

Closing the Security Activation Gap: A Security-by-Design Methodology for IoT Ecosystems and Its Empirical Validation in a DC Smart Microgrid

Raziel Campos-Sanchez^{a,b,*}, Luis Hernandez-Martinez^a, Claudia Feregrino-Urbe^{b,c}, Alejandro Penuelas-Angulo^{d,e}, Diego Cruz-Aguilar^{b,c}

^a*Electronics Department, National Institute of Astrophysics, Optics and Electronics (INAOE), Luis Enrique Erro 1, San Andrés Cholula, 72840, Puebla, Mexico*

^b*Science and Technologies of Security Department, INAOE, Luis Enrique Erro 1, San Andrés Cholula, 72840, Puebla, Mexico*

^c*Computational Sciences Department, INAOE, Luis Enrique Erro 1, San Andrés Cholula, 72840, Puebla, Mexico*

^d*CEIT-Basque Research and Technology Alliance (BRTA), Manuel Lardizabal 15, Donostia/San Sebastián, 20018, Guipuzcoa, Spain*

^e*Universidad de Navarra, Tecnum, Manuel Lardizabal 15, Donostia/San Sebastián, 20018, Guipuzcoa, Spain*

Abstract

A persistent security activation gap constrains the widespread adoption of IoT ecosystems: security mechanisms are widely available in commercial platforms yet remain inactive after standard deployment. Closing this gap requires not only improved tools but also a methodology that embeds cybersecurity as a foundational design concern from the earliest development stages. We present a security-by-design methodology for centralized IoT ecosystems that treats identity as an intrinsic entity property, spans seven stages across four phases, and integrates cybersecurity throughout the development life-cycle. A dynamic Root of Trust process establishes verifiable cryptographic identity for each entity, providing the trusted foundation for authentication, access control, and secure communication.

*Corresponding author

Email addresses: rcampos@inaoe.mx (Raziel Campos-Sanchez), luish@inaoe.mx (Luis Hernandez-Martinez), cferegrino@inaoe.mx (Claudia Feregrino-Urbe), apenuelasan@ceit.es (Alejandro Penuelas-Angulo), diego.cruz@inaoe.mx (Diego Cruz-Aguilar)

We validate the methodology through a confirmatory case study (Yin, 2014) applying all seven stages to the design and implementation of a DC Smart Microgrid (SMG): an industrial IoT ecosystem comprising two ESP32-based secondary control nodes and a Raspberry Pi 4B primary coordination agent. Four case study questions (CSQ1–CSQ4) evaluate guidance sufficiency, identity model unambiguity, Root of Trust verifiability, and threat coverage. Results show that all seven stages produced documented, traceable artifacts with no external guidance required (CSQ1); that a 14-parameter identity model across four entity types was produced without ambiguity (CSQ2); that the Root of Trust process established mutual authentication with a 56.7 ms total P2+P3 handshake latency (averaged across two independent bench runs), a per-message P4 latency of 30.3 ± 3.3 ms (196 pooled bench messages, NODE_02), and 100% chain integrity with zero security errors (CSQ3); and that the methodology achieves unweighted OWASP IoT Top 10 coverage of 7/10 items fully addressed and 3/10 partially addressed—I4 secure-update policy only, I8 device management without a cryptographically verifiable chain, and I10 physical hardening outside software scope (CSQ4). The CSQ3 figures above describe the case study’s original PSK+NVS Root of Trust instantiation; a subsequent PUF-based redesign (Section 4) was independently validated end-to-end on real hardware for one of the two SMG nodes ($N = 30$ hard-reset trials against a real WiFi access point and a real primary-agent server): 30/30 end-to-end success, mean mutual-authentication handshake latency 80.7 ± 36.2 ms, with PUF-response reconstruction requiring exactly two attempts in every trial (Section 6.4); validation of the second node, and full replacement of the CSQ3 figures above, remain future work. The methodology’s parameterized, technology-agnostic structure is designed to generalize across diverse IoT application domains; empirical validation in additional domains remains future work.

Keywords: Internet of Things, Security-by-Design, IoT Methodology, Root of Trust, Identity Management, Smart Microgrid, Case Study Research

1. Introduction

The Internet of Things (IoT) has evolved from a conceptual vision into a technological paradigm underpinning critical infrastructure, industrial automation, healthcare systems, and smart cities (Al-Fuqaha et al., 2015; Zhou et al., 2021). Despite this expansion, large-scale IoT adoption remains con-

strained by fragmented architectures, heterogeneous technologies, and—critically—the failure to integrate cybersecurity from the earliest design stages (Shahinzadeh et al., 2024; Makhdoom et al., 2018).

The consequences of this gap are well documented. The Mirai botnet, which compromised over 600,000 IoT devices to launch unprecedented distributed denial-of-service attacks, originated from misconfigured or unprotected deployments relying on default credentials (Antonakakis et al., 2017). Subsequent analyses confirm that cyberattacks on IoT infrastructure continue to escalate in frequency and sophistication, targeting energy systems, healthcare networks, and public utilities (Abraham et al., 2025; Okika et al., 2025).

A central observation motivating this work is that the principal barrier to secure IoT deployment is not the *absence* of security features but the *security activation gap*: the disconnect between security capabilities *available* in a platform and those *operational* after a standard deployment (Figure 1). Widely adopted open-source IoT platforms provide security mechanisms that require manual post-installation configuration, effectively restricting their use to organizations with dedicated security expertise (Fagan et al., 2021).

A second, more fundamental barrier operates at the methodological level: most IoT development projects lack a structured process that integrates cybersecurity from organizational decision-making through operational maintenance. Security is typically addressed *after* architectural and technology decisions are made, producing mechanisms that are bolt-on rather than foundational, with identity management that is platform-specific rather than entity-centric.

To address both barriers, we present two complementary contributions:

1. **A security-by-design methodology** for developing centralized IoT ecosystems. The methodology treats identity as an intrinsic property of every entity, spans seven stages across four phases (organizational strategy, architecture, implementation, and operation), and embeds cybersecurity—from threat policy to Root of Trust establishment—as a foundational concern from problem definition through ongoing operation. A dynamic Root of Trust (RoT) process establishes verifiable cryptographic identity for each entity before any operational communication begins.
2. **An empirical confirmatory case study** applying the methodology to the design and implementation of a DC Smart Microgrid (SMG)

IoT ecosystem, following Yin’s (2014) Case Study Research design. The SMG comprises two ESP32 secondary control nodes and a Raspberry Pi 4B primary coordination agent, reaching Technology Readiness Level 3. Four case study questions evaluate guidance sufficiency, identity model unambiguity, Root of Trust verifiability, and OWASP IoT threat coverage.

Conceptual foundation: The identity-centric IoT paradigm, the IoT/ecosystem distinction, the formal entity identity model, and the reconceptualization of the Root of Trust as a multi-phase process are established at the conceptual and definitional level in a companion manuscript (Campos-Sanchez et al., 2026). The present work builds directly on that foundation to specify, validate, and empirically evaluate an operative seven-stage development methodology; it does not re-derive these definitions independently.

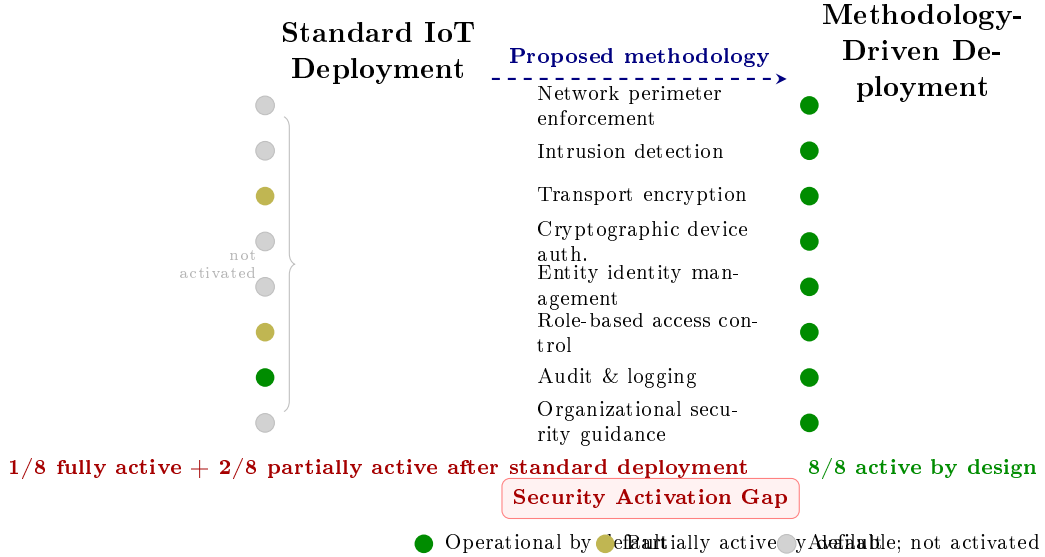


Figure 1: The security activation gap: standard IoT deployments activate only a subset of platform security controls by default; the remaining controls require manual post-installation configuration by practitioners with dedicated security expertise. The proposed methodology activates all controls by design, as a direct outcome of following the seven development stages.

Scope note: The ESP32/C (ESP-IDF) firmware suite and the primary agent software implementing the SMG are presented as part of this work’s own case

study rather than as a companion contribution under separate review. The present work evaluates the *methodology* through the case study; implementation details are provided only to the extent required for methodological evidence.

We evaluate the methodology through: (a) stage-by-stage traceability, demonstrating that all seven stages produce documented, actionable artifacts; (b) quantitative measurements of identity provisioning, handshake latency, sampling performance, and heap stability; and (c) a systematic OWASP (Open Web Application Security Project) IoT Top 10 coverage analysis.

The remainder of this article is organized as follows. Section 2 reviews related work. Section 3 presents the conceptual framework. Section 4 details the methodology and its STRIDE threat coverage model. Section 5 presents the SMG case study. Section 6 reports empirical results. Section 7 discusses findings, and Section 8 concludes.

2. Related Work

We adopt the definition of IoT methodology from Fortino et al. (2020): a structured set of guidelines, methods, practices, and documents organized within a logical framework for solving the specific problem of IoT ecosystem development. We evaluate existing methodologies against nine criteria: the eight classical dimensions drawn from the literature, supplemented by a ninth, *security validation*, which captures whether the methodology’s security provisions are supported by empirical evidence from a formal case study or experiment.

Table 1 compares representative methodologies. A 2023 comparative study (Moreta et al., 2023) confirms that cybersecurity integration remains the most underdeveloped dimension across IoT development methodologies. Most proposals emphasize functional aspects—interoperability, scalability, technological integration—while treating cybersecurity as an afterthought or omitting it entirely (Fortino et al., 2020; Al-Fuqaha et al., 2015; Merzouk et al., 2020). Agent-based approaches (Manate et al., 2014; Ayala et al., 2015; Spanoudakis and Moraitis, 2015) provide structured development processes but lack explicit security mechanisms and identity assignment. Model-driven methodologies (Morin et al., 2017; Costa et al., 2016; Cicirelli et al., 2017) address code heterogeneity and portability but do not consider entity identity or security mechanisms. A systematic review of agile methodologies

applied to IoT (Guerrero-Ulloa et al., 2023) confirms that even the most recent structured approaches prioritize sprint-based functional delivery, deferring authentication, identity management, and threat modeling to after functional requirements are met—if they are addressed at all.

Security-focused proposals, while noteworthy, remain limited in scope and depth. Josyula and Gupta (2017) presents a security methodology for IoT systems engineering yet defines processes at a general level without specific guidelines for identity assignment or cryptographic mechanism selection. Savaglio et al. (2018) includes security aspects (authorization, key management, authentication) in its reference architecture but does not address selection criteria or the timing of security controls relative to development stages. Muñoz et al. (2025) proposes a five-layer service-oriented architecture with security as a cross-cutting concern but does not explain how mechanisms are integrated or selected; its PKI-based scheme further neglects resource-constrained devices. A systematic review of IoT authentication schemes published in this journal (Saqib and Moon, 2023), covering over 120 proposals from 2010 to 2022, confirms the core fragmentation: authentication mechanisms are consistently proposed in isolation, without connection to a broader development process, formal identity model, or measurable threat coverage metric. Blanco et al. (2023) propose an ontology-driven framework for cyber-physical system security requirements modeling, providing structured elicitation but not a full development lifecycle spanning organizational policy through operational deployment.

More recently, several efforts have targeted specific IoT security domains, yet none provide holistic development guidance. Zero Trust architectures for industrial IoT (Zanasi et al., 2024; Fernandez and Brazhuk, 2024), blockchain-based supply chain security (Ray et al., 2024; Fernando et al., 2023), and SDN-based IoT fog security (Javanmardi et al., 2023; Turner et al., 2023) address important individual threat surfaces but do not integrate into a development methodology. Visintin (2024) examines identity of things from a device-centric perspective, identifying important device-level identity requirements, yet does not extend to ecosystem-wide identity management across heterogeneous entity types (devices, servers, users, administrators) or connect identity to authentication and access control policy. PUF-based Root of Trust implementations for constrained devices (Sassalou et al., 2025) and hardware security modules (Bakiris and Oikonomakis, 2025) strengthen device-level identity but remain single-layer solutions that leave the broader design and development process unaddressed. The present work adopts an

SRAM-based PUF implementation for the ESP32 microcontroller family, `esp32_puflib` (Stanicek, 2022), as the concrete Stage 4 technology instantiating identity-parameter generation without persistent on-device key storage (Section 4), integrating it into the full seven-stage development process rather than treating it as an isolated device-level mechanism.

Standards from government agencies and consortia—Secure-by-Design (UK Government Security, 2024), NIST IoT Cybersecurity Guidance (Fagan et al., 2021), ETSI EN 303 645 (European Telecommunications Standards Institute, 2020), the CSA IoT Security Controls Framework (Cloud Security Alliance, 2023), and the 2025 NIST cybersecurity supply chain risk guidance (Licata et al., 2025)—provide valuable security baselines. However, these standards remain predominantly prescriptive at the policy level, offering no technology selection criteria, no process for deriving implementation choices from organizational constraints, and no deployment automation.

Taken together, these approaches share a common limitation: the absence of a methodology that simultaneously addresses (a) organizational adoption strategy, (b) technology-agnostic architectural design with a formal entity-centric identity model, (c) a dynamic Root of Trust process establishing verifiable cryptographic identity, (d) iterative development guided by maturity levels, and (e) empirical validation through a formal case study with measurable artifacts at every stage. The methodology presented in this work addresses all five dimensions. The validation follows Yin’s (2014) confirmatory case study design with four research questions and documented stage-by-stage artifact traceability—we are not aware of a comparable empirical validation of a complete IoT security development methodology using this research design.

3. Conceptual Framework

Ambiguity in IoT definitions hinders coherent methodology design (Sorri et al., 2022). Rather than re-deriving reference definitions, this methodology builds directly on the identity-centric conceptual framework established in a companion manuscript (Campos-Sanchez et al., 2026), which reframes IoT as a technological paradigm in which cyber-physical systems are constitutively endowed with identity, and distinguishes this paradigm from an *IoT ecosystem*—a digital environment in which entities possessing unique identities interact under a defined structure to provide services that solve identified

Table 1: Comparison of IoT ecosystem development methodologies (9 criteria). Tech.Dev. = technological development guidance; CyberSec. = explicit cybersecurity integration; Agnostic = technology-agnostic design; Org. = organizational aspects; Scalability; Maturity = TRL or equivalent; Interop. = interoperability; Concept. = formal conceptual framework; Sec.Val. = security provisions validated through formal case study or experiment. ✓ full, ~ partial, ✗ absent. *This work* is rated ~ on Tech.Dev. because the methodology is technology-agnostic by design: it specifies mechanism categories and stage requirements, not specific technology selection. Ratings for “This work” are self-assessed by the authors.

Methodology	Tech. Dev.	Cyber Sec.	Agnostic	Org.	Scalability	Maturity	Interop.	Concept. Frame.	Sec. Val.
Manate et al. (2014) (Manate et al., 2014)	—	—	✗	✗	✓	✗	✓	✗	✗
Ayala et al. (2015) (Ayala et al., 2015)	✓	✗	✓	✗	—	✗	✓	—	✗
Spanoudakis et al. (2015) (Spanoudakis and Moraitis, 2015)	✓	✗	✗	—	✓	✗	✓	✗	✗
Patel et al. (2015) (Patel and Cassou, 2015)	✓	✗	✗	✗	✓	✓	✓	✓	✗
Costa et al. (2016) (Costa et al., 2016)	✓	✗	✗	✗	✓	✗	✓	✗	✗
Fortino et al. (2017) (Fortino et al., 2017)	✓	✗	✗	✗	✓	✓	✓	✗	✗
Josyula et al. (2017) (Josyula and Gupta, 2017)	✗	—	✓	✗	✓	✗	✓	—	✗
Savaglio et al. (2018) (Savaglio et al., 2018)	—	—	✓	✗	✓	✗	✓	—	✗
Casadei et al. (2019) (Casadei et al., 2019)	✓	✗	—	✗	✓	✗	✓	—	✗
Guerrero-Ulloa et al. (2023) (Guerrero-Ulloa et al., 2023)	✓	✗	✓	—	—	✗	—	✗	✗
Muñoz et al. (2025) (Muñoz et al., 2025)	✓	—	✗	✗	—	✗	—	—	✗
This work (2026)	—	✓	✓	✓	✓	✓	✓	✓	✓

problems. We adopt both definitions unchanged and use them as the entry point for the present methodology.

The ecosystem comprises seven element types: IoT devices (embedded systems with ecosystem-relative identity), IoT servers (network-connected infrastructure enabling entity communication), IoT services (processes enabling ecosystem operation and security), storage systems (information persistence infrastructure), users (business-logic-level entities with defined roles), administrators (operational management entities), and communication infrastructure (data exchange enablers).

Definition 1 (Cybersecurity in an IoT Ecosystem). *Intrinsic properties of the design, development, structure, integration, operation, and physical conditions that determine the vulnerability, robustness, resilience, and response to disturbances or malicious interventions in IoT ecosystems.*

We likewise adopt the reconceptualization of the Root of Trust (RoT) as a multi-phase *process*, rather than a static hardware component, established in Campos-Sanchez et al. (2026): entities establish, exchange, and securely store identity parameters within a trusted environment before any operational communication begins. This process accommodates heterogeneous device capabilities—from constrained microcontrollers to cloud servers—supports

both symmetric and asymmetric cryptographic schemes, and establishes the trusted context required for provisioning, authentication, and secure session establishment across all entity types. When Phase 1 (parameter generation) is instantiated through a Physically Unclonable Function (PUF) rather than random generation followed by persistent storage, Phase 2 (secure storage) is correspondingly trivialized: the entity’s private identity parameter is never stored at rest—only non-secret helper data required to reconstruct it deterministically—removing the secure-storage attack surface rather than merely hardening it. Stage 6.1.3 (Section 4) instantiates this process concretely for the case study’s entity types.

4. Proposed Methodology

The methodology is organized into four phases and seven stages (Figure 2). Each stage produces documented outputs that serve as inputs to the subsequent stage, and is itself decomposed into numbered sub-processes (e.g., 2.2.4, 6.1.3) that the case-study evidence cites throughout this article; the full numbered specification is provided in Appendix A.

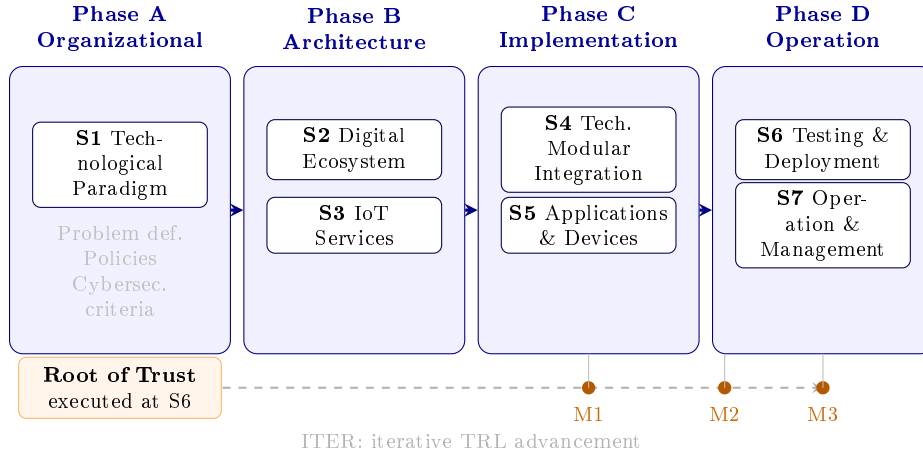


Figure 2: Organization of the proposed methodology: four phases (Organizational, Architecture, Implementation, Operation) comprising seven numbered stages (S1–S7). Iterative development cycles (ITER) advance Technology Readiness Level. Maintenance points M1 (implementation), M2 (deployment), and M3 (operation) trigger targeted stage re-entry. The Root of Trust is executed at Stage 6.

4.1. Phase A: Organizational (Stage 1)

This phase addresses organizational considerations for IoT adoption. Stage 1 establishes the problem to be solved, operating policies, development policies, and cybersecurity criteria. The key processes are: problem definition with cost-benefit analysis (1.1); operating policies covering functional objectives, financial constraints, and regulatory compliance (1.2); development policies defining an iterative strategy using the TRL framework, timelines, and resource allocation (1.3); and cybersecurity policies covering criticality assessment, technological sovereignty, data sovereignty, supply chain security, and applicable government guidelines including Secure-by-Design (UK Government Security, 2024) and Privacy-by-Design (European Union Agency for Cybersecurity, 2017) principles (1.4). Processes 1.4.2–1.4.3 operationalize the organizational-strategy layer of the sovereignty framework established in Campos-Sanchez et al. (2026), under which technological and data sovereignty are treated as emergent properties of who controls the entities’ Root of Trust process rather than as external policy overlays.

4.2. Phase B: Architecture (Stages 2–3)

Stage 2 defines the IoT ecosystem architecture at the entity level: integration diagram, communication topology, entity hierarchy, functional processes, data flow, and scalability strategy. Critically, Stage 2 instantiates the formal identity model established in Campos-Sanchez et al. (2026), under which each entity X in ecosystem E possesses an identity:

$$I_X^E = P_{X\text{ pub}}^E \cup P_{X\text{ priv}}^E \quad (1)$$

where public parameters $P_{X\text{ pub}}^E$ serve as identifiers and private parameters $P_{X\text{ priv}}^E$ enable cryptographic operations, and authentication parameters $P_{X\text{ auth}}^E \subset I_X^E$ and shared secrets $P_{X\text{ SS}}^E \subset P_{X\text{ auth}}^E$ are derived from this identity foundation (Figure 3). This methodology extends that model with an explicit cardinality count for each entity type X : the public-parameter and private-parameter counts are:

$$n_X^E = |P_{X\text{ pub}}^E| \quad (2)$$

$$k_X^E = |P_{X\text{ priv}}^E| \quad (3)$$

so that the total identity parameter count is $n_X^E + k_X^E$ and the table columns report the two component counts and their sum.

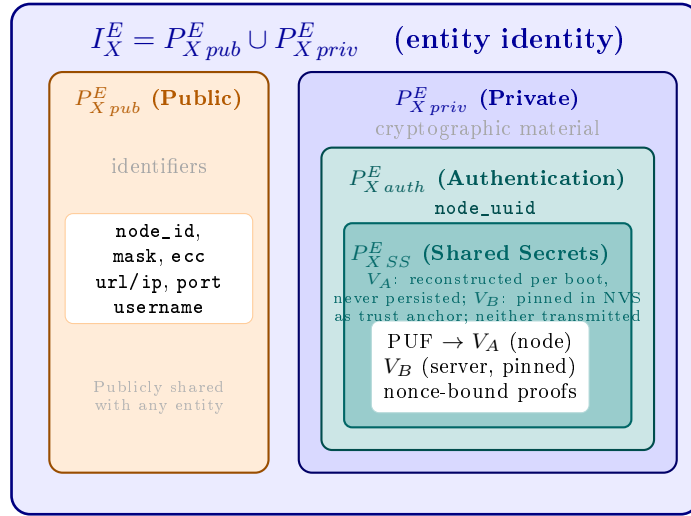


Figure 3: Identity parameter set hierarchy for entity X in ecosystem E . Shared secrets P_{SS} enable cryptographic operations; authentication parameters P_{auth} prove identity; private parameters P_{priv} are never disclosed; public parameters P_{pub} serve as identifiers. In the SMG, the node’s private identity parameter (its PUF response) lives in P_{priv} but is never persisted to storage—only the non-secret mask/ECC helper data used to reconstruct it lives in ESP32 NVS, as P_{pub} .

Stage 3 specifies the services required for entity operation: secure storage subsystems, access control mechanisms, identity parameter storage strategies, and regulatory compliance integration. For constrained devices, Stage 3 requires an explicit cost/risk evaluation of persistent secure-storage options (hardware security modules, encrypted flash partitions) against physically-derived alternatives that avoid persisting a secret altogether; a Physically Unclonable Function (PUF) that regenerates the private identity parameter on demand from non-secret helper data is one such alternative, and its selection at Stage 4 is a direct consequence of this Stage 3 evaluation rather than an independent design choice. Servers and users retain conventional secrets management and MFA, respectively.

4.3. Phase C: Implementation (Stages 4–5)

Stage 4 selects enabling technologies aligned with policies from Stage 1 and architecture from Stages 2–3. Technology choices are evaluated against maturity level (TRL), vulnerability landscape, and supply chain risk. For the SMG, Stage 4 selects `esp32_puflib` ([Stanicek, 2022](#)), an SRAM-based PUF implementation for the ESP32 family, as the identity-generation mechanism motivated by the Stage 3 secure-storage evaluation above. Because `esp32_puflib` is implemented as a native C/ESP-IDF component, Stage 4 also selects a C/ESP-IDF firmware architecture for the node—rather than the MicroPython runtime used in the prior PSK+NVS instantiation—so that the PUF library is linked directly, without a language-runtime wrapper; the same decision replaces the previous MicroPython `aesgcm` companion module with ESP-IDF’s native `mbedtls/esp_aes` AES-256-GCM support. This platform choice is itself a direct, traceable consequence of the Stage 3 evaluation, not an independent implementation preference. Stage 5 implements all ecosystem elements: servers, devices, databases, communication infrastructure, IoT services, and applications. Security is enforced through secure coding practices, unit testing of security mechanisms, and validation against applicable security guidelines (Secure-by-Design principles ([UK Government Security, 2024](#)), NIST IoT Cybersecurity Guidance ([Fagan et al., 2021](#)), and OWASP IoT Top 10).

4.4. Phase D: Operation (Stages 6–7)

Stage 6 covers deployment, configuration, and testing: individual component validation, infrastructure deployment, *Root of Trust execution* for all entities (Process 6.1.3), general operation verification, and security testing

(penetration testing at higher TRLs; tamper verification at TRL 3). Stage 7 addresses ongoing operation: monitoring, element maintenance, information lifecycle management, threat intelligence, element updates, incident response, and network infrastructure oversight.

4.5. *Iterative Development and Maintenance*

The methodology supports iterative advancement through TRL-guided cycles (ITER) and three structured maintenance points: M1 (during implementation, triggers technology re-evaluation), M2 (during deployment, triggers code revision), and M3 (during operation, triggers architectural redesign). M1 fires when an implementation-phase review reveals that the chosen technology cannot satisfy a stage requirement; M2 fires when deployment-phase testing reveals that the implementation requires code-level revision to meet a security or functional requirement; M3 fires when operation-phase monitoring reveals that the deployed architecture cannot accommodate an observed operational pattern. This structure supports progressive maturation from proof-of-concept to production deployment.

4.6. *Root of Trust Establishment*

Building on the four-phase Root of Trust process of [Campos-Sanchez et al. \(2026\)](#), we instantiate its parameter generation, secure storage, exchange, and peer-management phases as concrete protocol message sequences for both cryptographic schemes.

Symmetric scheme (resource-constrained devices): Rather than generating a random device key and persisting it in flash, the device’s identity key is derived on demand from a Physically Unclonable Function (PUF), instantiated via `esp32_puflib` ([Stanicek, 2022](#)). *Enrollment* (performed once, Table 2) reads the ESP32’s RTC FAST SRAM power-up state across repeated measurements and derives two non-secret artifacts—a stable-bit mask and error-correction (helper) data—that are stored on the device and allow the same physical response to be reconstructed, with correction, on every subsequent boot; neither artifact discloses the response without physical access to the specific chip. During the same enrollment, the device computes a one-way verifier of its response and registers it with the server; the server’s own verifier (its identity counterpart, not PUF-derived—see below) is pinned on the device. *Session identification* (Table 3) is a *cryptographic hash-reconstruction challenge*: each side issues the other a fresh random nonce, reconstructs its own verifier (the device physically, via the PUF; the server from conventional

storage), and proves fresh knowledge of it by hashing it together with the peer’s nonce—never transmitting the verifier or the underlying response itself, and never reusing a prior proof, since each is bound to a nonce that changes every session. Parameters are stored securely (Table 2); critically, the device persists no secret of its own: the only values written to its NVS are the non-secret stable-bit mask and ECC helper data (both classified P_{pub}) and the pinned server verifier V_B , which serves as the device’s trust anchor for server authentication. The node’s own private identity parameter (PUF_A , and hence its verifier V_A) exists only transiently in RAM during each boot and is never written to storage (Section 3).

Server identity: the server is not a good physical candidate for PUF-based identity (no comparable unclonable analog substrate is assumed for the RPi4), so its identity parameter is a conventionally-provisioned, equivalent counterpart in the same formal role: a value provisioned on the server and pinned on each enrolled device as that device’s server-authentication trust anchor. Because the scheme is symmetric, possession of V_B is exactly what the server must prove per session, via the same nonce-bound hash-reconstruction challenge the device applies to its own V_A ; equivalently, V_B functions as the pairwise shared secret for server authentication, and its confidentiality on the device side is bounded by the device’s physical security (see the Information Disclosure row in Table 6 and the Limitations). This keeps both entities’ identity parameters structurally symmetric in the model of Campos-Sanchez et al. (2026) ($I_X^E = P_{X_{pub}}^E \cup P_{X_{priv}}^E$) while acknowledging that only the constrained device benefits from a physically-unclonable generation mechanism.

Asymmetric scheme (capable devices): Entities generate public/private key pairs, exchange public keys with identifiers, and store the peer’s public key for subsequent authentication using ISO/IEC 9798-3 mutual authentication (ISO/IEC, 2019) (Table 4).

Implementation note: the PUF-based implementation source (`smg_puf_rot_node/`, device side; `smg_primary_server/rot_challenge_service.py`, server side; companion code repository, Materials Availability) uses `ShS_A/ShS_B` for V_A/V_B , `challenge_a/challenge_b` for N_A/N_B , and `server_proof/device_proof` for R_B/R_A ; the four-message sequence and the Key_{ab} derivation above match the implementation exactly, including concatenation order, verified directly against the source on 2026-10-05.

Table 2: Root of Trust process: PUF-based symmetric key establishment — Phase 1 (enrollment and mutual verifier registration, performed once, over an isolated/physical provisioning channel). PUF_A denotes the device’s PUF response, reconstructed from RTC FAST SRAM via `esp32_puflib`; $\text{mask}_A/\text{ecc}_A$ are the non-secret stable-bit mask and error-correction helper data. V_B is the server’s conventionally-provisioned identity counterpart (Section 4, “Server identity”), not PUF-derived.

Device	Message	Server
$\{ID_A, MAC_A\}$ <code>enroll_puf()</code> $\rightarrow \{\text{mask}_A, \text{ecc}_A\}$ (stored, public) <code>get_puf_response()</code> $\rightarrow \text{PUF}_A; V_A \leftarrow$ $\text{SHA-256}(\text{PUF}_A)$	$\{ID_A, MAC_A, V_A\} \rightarrow$ $\leftarrow \{ID_B, URL/IP_B,$ $Port_B, V_B\}$	$\{ID_B, URL/IP_B, Port_B\};$ V_B provisioned (stored)
pin (store): $\{ID_B, V_B\}$ as server reference		store verifier: $\{ID_A, V_A\}$ as device reference (not PUF_A itself)

Table 3: Root of Trust process: PUF-based symmetric key establishment — Phase 2 (per-boot reconstruction and nonce-bound hash-reconstruction challenge; continues Table 2). V_A is never transmitted after Phase 1—each side proves fresh knowledge of its own reconstructed verifier by hashing it with a nonce supplied by the peer. No secret is persisted on the device between Phase 1 and Phase 2; PUF_A is regenerated each time.

Device	Message	Server
generate nonce N_A get_puf_response() using $\{\text{mask}_A, \text{ecc}_A\}$ $\rightarrow \text{PUF}_A; V_A \leftarrow$ SHA-256(PUF_A) verify $R_B \stackrel{?}{=}$ SHA-256($V_B \parallel N_A$) using pinned V_B (server authentication); $R_A \leftarrow \text{SHA-256}(V_A \parallel$ $N_B)$	$\{ID_A, N_A\} \rightarrow$ $\leftarrow \{ID_B, N_B, R_B =$ SHA-256($V_B \parallel N_A\})\}$ $\{R_A\} \rightarrow$ $\leftarrow \{\text{session_ack}\}$ (or rejection if verification failed)	generate nonce N_B compute R_B from stored V_B verify $R_A \stackrel{?}{=}$ SHA-256($V_A \parallel N_B$) using stored reference V_A (device identification) issue session_ack only if R_A verified
SHA-256($V_A \parallel V_B \parallel$ $N_A \parallel N_B$) $\rightarrow \text{Key}_{ab}$ Key_{ab} held only in RAM for session duration; no on-device secure store required		SHA-256($V_A \parallel V_B \parallel$ $N_A \parallel N_B$) $\rightarrow \text{Key}_{ab}$ Key_{ab} held in RAM; V_A reference retained in server storage

Table 4: Root of Trust process: asymmetric key establishment.

Device	Message	Server
$\{ID_A, MAC_A, PubKey_A, PrivKey_A\}$	$\{ID_A, MAC_A, PubKey_A\}$	$\{ID_B, URL/IP_B, Port_B, PubKey_B, PrivKey_B\}$
	$\rightarrow$	
	$\leftarrow \{ID_B, URL/IP_B, Port_B, PubKey_B\}$	
secure store: $\{PubKey_B, ID_B, URL/IP_B, Port_B\}$		secure store: $\{PubKey_A, ID_A, MAC_A\}$

4.7. Threat Coverage by Design: STRIDE Analysis

We analyze how the methodology’s stages address the STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege) threat categories (Shostack, 2014) for centralized IoT ecosystems. Unlike framework-specific threat models tied to particular technologies, this analysis maps threats to methodology stages and technology-agnostic mechanism categories, ensuring applicability across implementations.

Tables 5 and 6 map each STRIDE threat to the methodology stage(s) providing countermeasures, the mechanism category mandated by those stages, and the instantiation in the SMG case study. The adversary $\mathcal{A}$ is assumed to have network access, commodity computational resources, and knowledge of the system architecture (Kerckhoffs’s principle); goals include unauthorized data access, service disruption, entity impersonation, and credential exfiltration.

5. Case Study: DC Smart Microgrid IoT Ecosystem

5.1. Case Study Design

This case study follows a **confirmatory design** (Yin, 2014): the methodology constitutes the theoretical proposition; the SMG implementation constitutes the empirical test. Evidence is collected from four sources following Yin’s principle of multiple evidence sources (Table 7).

Table 5: STRIDE threat model: mapping to methodology stages and SMG instantiation
— Part 1 (Spoofing, Tampering, Repudiation).

Threat	Attack Vector	Methodology Stage(s)	Mechanism Category	SMG Instantiation	Residual Risk
Spoofing	Entity impersonation; forged credentials	2.2.2 (auth mech.), 2.2.3 (RoT), 6.1.3 (RoT execution)	Cryptographic mutual authentication using entity-specific, PUF-derived identity commitments	PUF-derived identity (<code>esp32_puflib</code>) + nonce-bound SHA-256 hash-reconstruction challenge; no persistent secret in NVS; mutual handshake before any data accepted; bidirectional mutual authentication confirmed in 30/30 real hard-reset trials on SMG_NODE_01 (Section 6.4)	Low (requires physical device compromise)
Tampering	Data modification in transit; sensor reading manipulation	2.2.4 (secure comms), 5.2.1 (secure dev.)	Authenticated encryption with integrity binding per message; random nonces	AES-256-GCM per message; 12-byte random nonce per message; the GCM authentication tag detects any single-message modification	Low for per-message tampering (GCM tag); Medium for message reordering/replay within a session (no message-sequence chain in this design, see Limitations)

Table 6: STRIDE threat model (continued from Table 5) — Part 2 (Information Disclosure, Denial of Service, Elevation of Privilege).

Threat	Attack Vector	Methodology Stage(s)	Mechanism Category	SMG Instantiation	Residual Risk
Information Disclosure	Key extraction; credential exfiltration; traffic interception	1.4.2–1.4.3 (sovereignty), 2.2.1 (identity params), 5.2.1 (secure dev.)	Data sovereignty policy; no persistent device-own secret to disclose; application-layer encryption	The node’s private parameter is reconstructed at boot from the PUF (<code>esp32_puflib</code>) and never persisted: only non-secret mask/ECC helper data is written to NVS, plus the pinned server verifier V_B (trust anchor); all data AES-256-GCM encrypted in transit	Node identity: Low (no node secret exists at rest; forging V_A requires the enrolled chip’s physical response or the server’s stored verifier). Server impersonation from a captured unit: Medium (V_B is readable from that unit’s NVS; the centralized single-primary topology contains the exposure — see Limitations)
Denial of	Authentication	7.1	Anomaly-based	Trust score decay	Medium

Table 7: Data collection protocol: evidence sources.

Source	Evidence Type	Content
Design documents	Documentary	Stage outputs: policies, architecture diagrams, identity tables, service specifications
Firmware artifacts	Physical artifact	ESP32 C/ESP-IDF modules: sensor reading, EMS, identity (PUF enrollment/reconstruction, <code>esp32_puflib</code>), communication (AES-256-GCM), metrics
Execution logs	Archival records	Authentication logs, RoT deployment logs, functional test reports, <code>/methodology.csv</code> output
Measurements	Quantitative	Handshake latency, sampling rate, payload size, heap stability, OWASP coverage

5.2. Research Questions

5.3. Case Context

Application domain: The SMG manages distributed DC electrical energy generation, storage, and consumption across two co-located nodes in a controlled laboratory environment. The growing integration of renewable energy sources (solar panels, batteries) into local grids requires intelligent, automated management to balance generation and demand in real time without relying on centralized national grid infrastructure. This application is classified as critical infrastructure (Methodology Stage 1, Process 1.4.1), requiring the highest cybersecurity tier.

Physical architecture: The SMG consists of two secondary control agent nodes and one primary agent (Raspberry Pi 4B). Each secondary node connects a renewable generation element to a shared 12 V DC bus with local energy storage (Table 9). The primary agent runs on a Raspberry Pi 4B connected to the DC bus as a pure load via a DC/AC inverter (12 V DC $\rightarrow$ 120 V AC) followed by a USB-C AC/DC adapter.

Digital architecture: The ecosystem uses a centralized communication topology (Stage 2.1.4) with fog processing (Stage 2.1.7): each ESP32

Table 8: Case study research questions.

ID	Question	Addressed by
CSQ1	Does the methodology provide sufficient guidance to architect a complete, secure IoT ecosystem for a real industrial application?	Stage-by-stage traceability; completeness of outputs; functional test results
CSQ2	Does the parameterized identity model enable unambiguous security mechanism definition across heterogeneous entity types?	Identity parameter table; authentication implementation; coverage across 4 entity types
CSQ3	Does the Root of Trust Process establish verifiable secure communication between ecosystem entities?	RoT execution evidence; handshake latency measurements; mutual challenge-response verification
CSQ4	What is the methodology’s coverage of known IoT security threats relative to established standards?	OWASP IoT Top 10 mapping; security mechanism traceability

Table 9: Secondary node hardware components.

Component	Role	Specification
ESP32 NodeMCU	IoT + EMS controller	Dual-core 240 MHz, 520 KB SRAM, 4 MB Flash, built-in WiFi; powered by local battery via DC-DC buck (12V→5V)
Epcom AGM VRLA	Local energy storage + ESP32 supply	PL-18-12, 12 V, 18 Ah; also supplies ESP32 via buck converter
IBFIA DC-DC SSR ×3	Energy flow control	3–32 VDC control, 5–60 VDC load, 40 A; driven by ESP32 LEDC on GPIO27/26/25 at 1 kHz, 10-bit
Schneider MCB ×3	Overcurrent protection	2-pole, one per SSR channel
Solar charge controller	Energy flow control	Commercial MPPT controller (handles droop control and battery-charging voltage regulation autonomously)
ACS712-05B ×2	Current sensing	Hall-effect, 185 mV/A, 16-sample median oversampling
Resistive divider ×2	Voltage sensing	30 k Ω /7.5 k Ω (5:1 ratio); reads up to 16.5 V on 3.3 V ADC

executes local sensor fusion and a 9-rule fuzzy Energy Management System (EMS), while the RPi4 runs global coordination with a 27-rule security-aware EMS. The entity hierarchy is: IoT Nodes (lowest privilege) < Users < Administrators. Development target: TRL 3 (proof of concept with physical elements).

5.4. Stage-by-Stage Traceability

Tables 10 and 11 document the application of each methodology stage to the SMG, showing the decision made at each stage and the measurable artifact produced. This traceability table constitutes the primary evidence for CSQ1.

5.5. Identity Parameter Definition (Stage 2.2.1)

Applying Equations (1)–(3) to the SMG entities produces a 14-parameter identity model (Table 12). The public/private/authentication parameter separation directly determined the authentication mechanism (PUF-derived commitment using the private, non-persistent parameter) and the secure storage requirement for the IoT Node: none, beyond the non-secret mask/ECC helper data already classified as public.

The entity-specific separation of identity parameters produced a direct, unambiguous mapping to implementation artifacts: `node_id`, `node_uuid`, `maskA`, and `eccA` correspond to distinct NVS keys written at enrollment time; no parameter redefinition or additional design decisions were required at the specification level. In the present case study, P_{auth} and P_{priv} coincide for every entity type (all private parameters participate in authentication, as shown in Table 12); this is a case-study-specific coincidence, not a general property of the model—Campos-Sanchez et al. (2026) define only the broader relation $P_{auth} \subset I$ (Figure 3), so future case studies may include a private parameter not used for authentication (e.g., a long-term audit key), in which case $P_{auth} \subsetneq P_{priv}$ rather than $P_{auth} = P_{priv}$.

[†]**Note on the PUF response as a non-persistent private parameter:** the formal identity model $I_X^E = P_{X_{pub}}^E \cup P_{X_{priv}}^E$ (Section 4) classifies a parameter as private or public by its confidentiality role, not by whether it is stored. The IoT Node’s PUF response satisfies P_{priv} ’s role—it is never disclosed and underlies the device’s authentication proof—while never existing as data at rest: it is regenerated from `maskA`/`eccA` (both P_{pub}) on every boot. This is the concrete instance of the Phase 2 (secure storage) trivialization noted in Section 3.

Table 10: Stage-by-stage traceability: methodology process $\rightarrow$ SMG decision $\rightarrow$ artifact
— Part 1 (Stages 1–3).

Stage	Methodology Process	Decision Made in SMG	Artifact
1 Technological Paradigm	1.1 Establish problem	Manage distributed renewable energy nodes securely in a 12 V DC microgrid	Problem specification
	1.1.2 Entities involved	5 entity types: IoT nodes, server, users, admins, DC bus	Entity list
	1.4.1 Criticality	Classified as critical infrastructure (electrical energy)	High-security cybersecurity policy
	1.4.2 Technological sovereignty	Self-hosted; no third-party cloud; ESP32 + local RPi4 server	Architecture constraint record
<i>Of the seven conceptual-framework element types (Section 3), the SMG instantiates five: storage/communication infrastructure are subsumed into the server, and IoT services into the device/server software. Of the five entity types, four (nodes, server, users, admins) carry digital identity (Table 12); the DC bus is tracked via P3 sensor-data integrity instead.</i>			
2 Digital Ecosystem	2.1.4 Communication topology	Centralized: all nodes report to primary agent	Architecture diagram
	2.1.7 Processing topology	Fog: local EMS on ESP32; global coordination on RPi4	Processing topology record
	2.2.1 Identity definition	4 entity types, 14 identity parameters total (non-uniform per type; Table 12)	Identity parameter table
	2.2.2 Authentication mechanism	Nonce-bound SHA-256 hash-reconstruction challenge (PUF-derived device identity, conventionally-provisioned equivalent server identifier); AES-256-GCM session encryption	Authentication diagram
	2.2.3 Root of Trust process	4-phase RoT designed for node–server pairs	RoT execution log
3 IoT Services	3.1 Service definition	4 services: data ingestion, EMS coordination, auth API, monitoring	Service specification
	3.2.1 Access control	Token-based; nodes: POST only; users: GET; admins: full	Access control matrix

Table 11: Stage-by-stage traceability (continued from Table 10) — Part 2 (Stages 4–7).

Stage	Methodology Process	Decision Made in SMG	Artifact
4 Tech. Modular Integration	4.1.1 TRL planning	TRL 3 target for ITER 1; laboratory proof of concept	TRL milestone record
	4.1.2 Platform selection	ESP32 + C/ESP-IDF: WiFi built-in, SHA-256 HW, NVS, <code>esp32_puflib</code> linked as a native ESP-IDF component	Technology selection rationale
	4.1.3 Communication	WiFi IEEE 802.11 b/g/n + TCP; application-layer hash-reconstruction challenge protocol (<code>rot_session.c/rot_challenge_service.py</code>)	Protocol selection record
	4.2 Vulnerability mitigation	ESP-IDF v5.4.2 pinned (per the flashed firmware build log); nonce reuse fixed	Risk register
5 Applications & Devices	5.1.1 Server	REST API + time-series database + monitoring dashboard on RPi4	Server deployment
	5.1.2 Devices	ESP32 C/ESP-IDF firmware (<code>smg_puf_rot_node</code>): identity (PUF), Root of Trust, and secure-communication modules; sensor/EMS modules not yet re-reported from the prior MicroPython firmware (see Limitations)	Firmware artifact
	5.2.1 Secure development	NVS stores only non-secret mask/ECC helper data (no persistent secret); <code>esp_fill_random()</code> nonces (ESP-IDF hardware RNG); no credentials in source	Code review record
6 Testing & Deployment	6.1.3 RoT execution	Prior PSK+NVS scheme: RoT executed for all 2 nodes at deployment (100%); PUF-based redesign: executed and validated on 1/2 nodes (<code>SMG_NODE_01</code> ; <code>SMG_NODE_02</code> pending, Section 6.4)	Deployment log (2 entries); PUF bench (Section 6.4)
	6.2.1 Functional testing	8 functional tests: boot, identity, auth, fuzzy EMS, saturation, all pass	Test report (8/8 pass)

Table 12: Identity parameter table: Stage 2.2.1 output for the SMG. $\text{mask}_A/\text{ecc}_A$: non-secret stable-bit mask and error-correction helper data generated at enrollment by `esp32_puflib`, stored in ESP32 NVS. PUF response[†]: the device’s physically-derived private identity parameter, reconstructed from $\text{mask}_A/\text{ecc}_A$ on every boot and never persisted (see note below). Following Eqs. (2)–(3), $n = n_X^E$ is the public-parameter count, $k = k_X^E$ the private-parameter count, and Total = $n_X^E + k_X^E$ the total identity-parameter count.

Entity Type	Class	P_{pub}	P_{priv}	P_{auth}	n	k	Total
IoT Node (ESP32)	Device	<code>node_id</code> , <code>mask_A</code> , <code>ecc_A</code>	<code>node_uuid</code> , PUF response [†]	<code>node_uuid</code> , PUF response [†]	3	2	5
IoT Server (RPi4)	Server	<code>url/ip</code> , <code>port</code>	<code>server_uuid</code>	<code>server_uuid</code>	2	1	3
User	User	<code>username</code>	<code>user_uuid</code> , <code>password</code>	<code>user_uuid</code> , <code>password</code>	1	2	3
Administrator	Admin	<code>username</code>	<code>admin_uuid</code> , <code>password</code>	<code>admin_uuid</code> , <code>password</code>	1	2	3
Total		7	7	7			14

5.6. Security Mechanism Analysis: STRIDE-to-Protocol Mapping

The methodology-level STRIDE analysis (Section 4.7) identifies mechanism categories that address each threat category at the framework level, independent of implementation technology. This subsection demonstrates how the SMG’s Stage 4 technology selection instantiates those mechanisms through three authentication and data-exchange protocol phases (P1–P3), establishing a complete traceability chain from threat identification to deployed countermeasure.

Scope note: Phases P1–P3 constitute an *implementation-specific* Stage 4 technology choice for the SMG (see Introduction scope note). Any protocol providing equivalent mechanism categories—pre-deployment identity isolation (P1), nonce-bound mutual identification without key transmission (P2), and per-message authenticated encryption (P3)—would satisfy the same STRIDE and OWASP properties against the same methodology requirements. This design intentionally omits a cryptographic session-chain/audit mechanism used in an earlier, since-discarded instantiation; Tables 5/6 and the Limitations (Section 7) record the resulting Repudiation/I8 trade-off explicitly.

Table 13 describes each phase’s security role, mechanism, and STRIDE/OWASP coverage. Table 14 summarizes the STRIDE coverage matrix across phases.

Spoofing is countered at two independent layers: P1 ensures that no entity can present a valid identity without physical access to the specific enrolled chip, since the identity is physically grounded rather than a copyable stored secret; P2’s nonce-bound hash-reconstruction challenge requires fresh, per-session demonstration of knowledge of each side’s verifier without revealing the underlying physical response or the verifier itself, confirmed bidirectionally before any session key is derived.

Tampering is detected at message creation time by the AES-256-GCM authentication tag on every message (P3); P2’s mutual identification additionally binds the session key to both parties’ proven identities, making a substituted message cryptographically detectable at the point of receipt. Unlike an earlier, since-discarded design, this scheme has no session-level hash chain, so retrospective, chain-based re-verification of message ordering is not available (secondary contribution downgraded to ~ in Table 14; see the Limitations, Section 7).

Repudiation coverage in this design is evidentiary rather than cryptographic: P3 messages are authenticated in transit, and the server logs each

Table 13: SMG authentication protocol phases: security role, mechanism, and coverage mapping (P1–P3). STRIDE: S = Spoofing, T = Tampering, R = Repudiation, ID = Information Disclosure, DoS = Denial of Service, EoP = Elevation of Privilege.

Ph.	Name	Security Role	Mechanism	STRIDE	OWASP
P1	Enrollment	Identity establishment before any network exposure	<code>enroll_puf()</code> (<code>esp32_puflib</code> (Stanicek, 2022)): 20 RTC-FAST-SRAM measurements $\rightarrow$ stable-bit mask + $8\times$ -repetition-code ECC helper data (both non-secret), written to ESP32 NVS via USB serial on isolated workstation; <code>node_uuid</code> (UUID-4), <code>wifi_ssid</code> , <code>wifi_pass</code> also written; verifier V_A registered with the server; no raw PUF response is ever written to flash	S, ID	I1, I9
P2	Root of Trust	Per-boot response reconstruction and nonce-bound mutual identification without key transmission	<code>get_puf_response()</code> reconstructs PUF_A (measured 418–421 bytes across 5 independent enrollments, 420 bytes reproducibly across $N = 30$ reconstructions of one fixed enrollment, on an SMG node’s physical ESP32-D0WD-V3 rev 3.1; above the ~ 380 –400-byte figure the library’s own design notes estimate for its reference hardware at the same stable-bit-probability setting, a chip-to-chip variation discussed in Section 7.1) from the stable-bit mask and ECC data; $V_A \leftarrow \text{SHA-256}(\text{PUF}_A)$; each side proves fresh knowledge of its own verifier by hashing it with a nonce supplied by the peer (Table 3); $Key_{ab} \leftarrow \text{SHA-256}(V_A \parallel V_B \parallel N_A \parallel N_B)$; neither the raw PUF response nor either verifier is ever transmitted; end-to-end mutual identification confirmed in	S	I1, I3

Table 14: STRIDE coverage matrix across protocol phases. ✓ = primary countermeasure (first line of defense); ~ = secondary/partial contribution; — = not applicable.

STRIDE Threat	P1	P2	P3
Spoofing	✓	✓	—
Tampering	—	~	✓
Repudiation	—	—	~
Info. Disclosure	✓	✓	✓
Denial of Service	—	~	—
Elev. of Privilege	✓	✓	—

session under `node_id` and timestamp (Stage 7.1), but—unlike the discarded hash-chain audit trail—there is no independently-verifiable cryptographic chain binding individual messages to a session that a third party could re-verify without trusting the server’s logs. This is an explicit, acknowledged trade-off for a smaller on-device footprint (see Limitations).

Information Disclosure is addressed at the source (P1: no key ever exists in storage—plaintext or otherwise—only non-secret helper data), during derivation (P2: the physical response is reconstructed transiently in RAM and never transmitted, and neither verifier is ever sent over the network), and in transit (P3: per-message AES-256-GCM with random nonces preventing ciphertext pattern analysis).

Denial of Service is partially addressed at P2: the nonce-bound challenge-response requires proof of verifier knowledge before the server commits further session resources, imposing a per-entity authentication cost on the attacker and preventing unauthenticated connections from consuming server session resources. Volumetric physical-layer DoS (wireless jamming, DC bus disruption) and application-layer flooding beyond the identification gate remain outside the scope of P1–P3. These are addressed at the methodology level by Stage 7 anomaly monitoring and trust-based isolation (Table 6, DoS row): nodes with trust score < 0.30 are disconnected immediately without waiting for a protocol timeout.

Elevation of Privilege is prevented at P1 (entity type embedded in provisioned identity) and enforced at P2 (entity type is a constituent of the identification exchange; all API access decisions in P3 are bound to the ses-

sion key established only after successful P2 identification).

6. Empirical Evaluation

6.1. CSQ1: Architecture Guidance Sufficiency

Traceability completeness: The stage-by-stage traceability tables (Tables 10 and 11) indicate that all 7 stages produced documented, verifiable artifacts—14 distinct outputs across the four phases. No stage required design decisions beyond what the methodology’s processes prescribe.

Functional validation: Table 15 reports the functional test results. All 8 tests passed across both nodes at first deployment attempt.

Iterative development confirmation (historical, prior PSK+NVS/MicroPython scheme): Three maintenance moments were identified and resolved during Stages 5–6 of the earlier SMG implementation, confirming that the methodology’s maintenance pathways operate as specified at TRL 3: (i) incorrect HKDF byte slice in P3 (M2: code revision); (ii) static nonce in session confirmation, where MicroPython’s default random module did not meet the methodology’s cryptographic randomness requirement, triggering re-evaluation of the RNG approach (M1: technology re-evaluation); and (iii) incorrect NVS hex encoding in provisioning (M2: code revision). Each was identified through the methodology’s systematic review requirements before deployment. The M3 (architectural redesign) mechanism was not triggered by any TRL 3 incident in this case study.

Iterative development confirmation (PUF-based redesign): One maintenance moment was identified and resolved during real-hardware bench validation of the PUF-based redesign on SMG_NODE_01 (Section 6.4): an enrollment-state-confusion bug, in which the firmware could not distinguish “never enrolled” from “recovering from a failed reconstruction attempt” (both read as the same transient library state), caused an unbounded re-enrollment loop whenever a reconstruction attempt failed its threshold check; it was fixed by basing the retry decision on the NVS-persisted enrollment flag rather than the library’s in-memory provisioning state (M1: technology/implementation re-evaluation), before the $N = 30$ trial reported in Table 19 was run. This single incident confirms the methodology’s M1 maintenance pathway operates as specified for this redesign as well; it is not a claim of a maintenance-cycle rate comparable to the three historical incidents above, and further incidents may surface once SMG_NODE_02 is validated.

Table 15: Functional test results (Stage 6.2.1; prior PSK+NVS/MicroPython instantiation). All tests executed on both nodes of that instantiation; the table is retained as historical evidence and has not been re-run under the PUF-based C/ESP-IDF redesign, whose own real-hardware validation is reported separately (Section 6.4). FT-01/FT-03–FT-05 reference that prior scheme’s mechanisms (`node_key`, HKDF, hash chain); FT-06–FT-08 exercise the sensor/EMS modules not yet re-ported to the redesign.

ID	Description	Result
FT-01	Boot: NVS identity loaded correctly (<code>node_id</code> , <code>node_key</code> , <code>node_salt</code>); boot time < 5 s	Pass (2/2)
FT-02	WiFi from NVS: connects using NVS credentials; no credentials in <code>config.json</code>	Pass (2/2)
FT-03	P2 RoT handshake: HKDF-SHA256 completes; ShS_{AB} established	Pass (2/2)
FT-04	P3 session: PSK handshake; SES_{key} established; AES-GCM confirmation verified	Pass (2/2)
FT-05	P4 hash chain: 1000 consecutive messages (unit test, separate from bench trial); chain integrity OK; 0 auth errors	Pass (2/2)
FT-06	Fuzzy EMS: all 9 rule combinations produce expected duty cycle ranges	Pass (1/1)
FT-07	ADC saturation flag: <code>SensorReading.saturated = True</code> when $V_{ADC} \geq 3.2$ V or ≤ 0.1 V	Pass (1/1)
FT-08	Degraded mode: last PWM maintained; safety limits applied; exponential backoff reconnect	Pass (1/1)
Overall pass rate		8/8 = 100%

Answer to CSQ1: The methodology provided sufficient guidance to produce a complete, operational IoT ecosystem. All 7 stages produced documented outputs with no external guidance required.

6.2. CSQ2: Identity Model Unambiguity

Applying the identity model equations to 4 entity types produced 14 distinct parameters with zero overlap between entity classes (Table 12). The public/private/authentication tripartite structure directly determined: (a) what is public non-secret helper data stored in NVS (mask_A , ecc_A , node_id), (b) what is never persisted and is instead reconstructed on every boot (the PUF response), and (c) what constitutes the authentication proof (node_uuid + the reconstructed PUF response). Table 16 summarizes these identity model metrics.

Table 16: CSQ2 identity model metrics.

Metric	
Entity types defined	
Total identity parameters	
Public parameters (P_{pub})	
Private parameters (P_{priv})	
Authentication parameters (P_{auth})	
Nodes provisioned via RoT	2/2 (100%) under the prior PSK+NVS scheme; 1/2 under the new scheme
Parameter collisions (overlapping definitions)	
Additional design decisions required at Stage 5	

Answer to CSQ2: The identity model is sufficient and unambiguous. All 4 entity types received complete, non-overlapping identity definitions that translated directly to implementation artifacts without modification.

6.3. CSQ3: Root of Trust Verifiability

The 4-phase RoT process was executed for both nodes at deployment. Post-provisioning, node authentication was verified by deliberate NVS parameter tampering—corrupted parameters consistently produced authentication failures, confirming that the RoT creates a verifiable security boundary.

Tables 17 and 18 report security performance measurements from independent bench trials on both SMG_NODE_02 (two trials, 196 pooled

bench sends) and SMG_NODE_01 (one recovery trial, 120 bench sends), running the same firmware. Each trial initializes the full protocol stack from boot and executes authenticated data sends (20 warm-up, then bench sends with the firmware guaranteeing at least one P4 send per second, i.e., a 1 s maximum inter-send interval) over the SMG_Primary WiFi AP; timing is exported via `metrics.py`. NODE_02’s two trials yielded 108 and 88 successful bench sends respectively (196 pooled), terminating early because the host-side `mpremote` console disconnected, triggering the P5 session-teardown path. NODE_01’s trial comprised 120 bench sends (post-calibration recovery run). V_{bus} , tracking error, and node power rows report physical-SMG measurements from both nodes; the EMS setpoint was fixed at 0 W (no active bus regulation), so the reported V_{bus} reflects the bus at float voltage rather than closed-loop 12 V tracking.

Answer to CSQ3: The RoT process established verifiable mutual authentication across both nodes (Table 17). The P2+P3 handshake completed in 56.7 ms (NODE_02, avg 2 runs) and 57.6 ms (NODE_01); the handshake is dominated by WiFi round-trip latency rather than cryptographic computation (HKDF + AES-256-GCM < 1 ms). P4 per-message latency averaged 30.3 ± 3.3 ms (NODE_02, 196 msgs) and 33.0 ± 4.1 ms (NODE_01, 120 msgs). The root cause of this latency is not established for the ESP32 platform: a companion PC-client (CPython) bench trial against the same server, run before the ESP32 firmware fixes, attributed its own P4 latency to server-side data-store lock contention from the primary agent’s background loops (uncontended baseline ≈ 1.7 ms on that client). Whether the same mechanism explains the ESP32-reported P4 latency above is unconfirmed — the ESP32 std (3.3/4.1 ms) is far tighter than the bimodal contended/uncontended split the PC-client diagnosis predicts, and no per-message ESP32 timestamps were retained to check for that pattern directly. Hash-chain integrity was 100% with zero security errors on both nodes; send success rate was 100%. Boot heap was consistent across nodes (NODE_02: 120,632 B; NODE_01: 122,672 B). The P5 latency (13.9/14.4 ms) reflects a firmware update that triggers chain archive and session-key teardown on every graceful exit.

6.4. PUF-Based Root of Trust: Real-Hardware Validation (SMG_NODE_01)

The preceding CSQ3 answer reports the case study’s original PSK+NVS/MicroPython bench trials (Tables 17–18). Following the Stage 3/4 redesign to the PUF-based scheme described in Section 4, the resulting C/ESP-IDF firmware (`smg_puf_rot_node`) was flashed to, enrolled on, and bench-validated against

Table 17: CSQ3: Root of Trust and protocol timing performance measurements — Part 1 (protocol/timing). NODE_02: pooled data from 2 bench trials (196 authenticated bench sends). NODE_01: 1 bench trial (120 authenticated bench sends). Both nodes: ESP32-D0WD-V3 rev 3.1, MicroPython 1.22.1, same firmware. All trials: 20 warm-up, maximum 1 s inter-send interval (firmware guarantees at least one P4 send per second). *Pooling convention*: for NODE_02, scalar means (P2, P3, handshake, P4 avg, sensor/EMS/loop means, heap) are the mean of the two run values; P4 std, EMS std, sensor std, and node-power std are the mean of the two within-run stds; V_{bus} std is the session-to-session (between-run) std of the two run means. *Provenance*: “Chain integrity 100%” and “Security errors 0” (Table 18) derive from the server-side P6 audit log, not from per-node CSV export.

Metric	NODE_02	NODE_01	Notes
Actual sampling rate (Hz)	2.73	2.72	Effective P4 send rate; control loop runs at ≈ 3.9 Hz (1/254 ms)
Loop avg $\pm$ std (ms)	253.7 ± 112.5	253.8 ± 113.4	Min: 172.7/172.8 ms; Max: 430.0/429.3 ms (N2/N1)
Sensor read avg $\pm$ std (μ s)	$171\,000 \pm 1\,751$	$170\,529 \pm 1\,164$	16-sample median oversampling
EMS inference avg $\pm$ std (μ s)	$1\,332 \pm 600$	$1\,340 \pm 660$	9-rule Mamdani centroid
P2 RoT latency (ms)	19.5	22.4	HKDF-SHA256 + 1 WiFi RTT (N2: avg 2 runs)
P3 session latency (ms)	37.1	35.1	PSK handshake + 2 WiFi RTTs (N2: avg 2 runs)
Handshake total (ms)	56.7	57.6	P2 + P3 combined; computed from raw μ s data (rounded components may not sum exactly)
P4 per-message avg $\pm$ std (ms)	30.3 ± 3.3	33.0 ± 4.1	AES-256-GCM + chain (N2: 196 msgs; N1: 120 msgs)
P5 logout (ms)	13.9	14.4	Triggers session key teardown and chain archive
Data payload avg (bytes)	165.3_{34}	166.2	JSON + P4 ciphertext
Bench-send chain length	196	120	Successful bench sends (excl. 20 warm-up)
Send success rate	100%	100%	All bench sends authenticated
Chain integrity	100%	100%	P6 hash-chain audit

Table 18: CSQ3: Root of Trust and security performance measurements (continued from Table 17) — Part 2 (heap and physical bus measurements). Heap rows: NODE_02 pooled from 2 bench trials; NODE_01 from 1 recovery trial. V_{bus} and node power rows: independent bench trials on both nodes; EMS setpoint fixed at 0 W (no active bus regulation).

Metric	NODE_02	NODE_01	Notes
Heap at boot (B)	120 632	122 672	Before module imports (MicroPython 1.22.1)
Heap at session (B)	116 008	118 048	Post-handshake
Free heap at shutdown (B)	89 232	84 640	<code>gc.mem_free()</code>
Heap drift per 100 loops (B)	−7 632	−7 344	Per-100-loop accumulation ($\sim 6.6\%$ / 100 loops); implies ~ 6 min sustained-runtime ceiling (see Limitations, S1)
Security errors	0	0	Auth + chain failures; per server-side P6 audit log (not in per-node CSV)
V_{bus} mean $\pm$ std (V)	13.598 ± 0.125	13.813 ± 0.038	N2: 2-run avg (session-to-session std); N1: post-calibration run_rec (within-run std); bus at float voltage
V_{bus} tracking error (V)	1.598	1.813	$ \text{mean} - V_{bus, \text{nominal}} $; bench setpoint at 0 W
Node power (W)	52.150 ± 1.624	43.632 ± 2.173	N2 sign-corrected (sensor polarity patch applied at reporting; source CSVs retain negative values); N2 std = mean of within-run stds

one of the SMG’s two physical nodes (SMG_NODE_01, ESP32-D0WD-V3 rev 3.1), connected over the real SMG_Primary WiFi access point to the real RPi4 primary agent (rpi4-server), running the server-side counterpart `rot_challenge_service.py`. **This complements, and does not yet replace, the CSQ3 figures above: validation to date covers one node, not both, and the original PSK-based measurements remain the only case study evidence for SMG_NODE_02.**

Table 19 reports results from $N = 30$ independent hard-reset trials against a single, fixed enrollment (the stable-bit mask and ECC helper data were generated once via `enroll_puf()` and not regenerated between trials); each trial therefore measures per-boot reconstruction-and-handshake reliability, not enrollment-to-enrollment variability, and exercises Phases P2 and P3 of the protocol (Section 5.6) against the real server on every reset, without repeating P1 enrollment. An earlier internal $N = 30$ round against the same enrollment, run before a firmware logging fix (removal of a diagnostic serial print of the raw corrected verifier, kept only for bring-up re-registration after an NVS reflash) and a cosmetic boot-message fix, produced consistent success/attempt-count figures but is superseded here by this more recent round, which matches the firmware as currently archived.

Three findings stand out. First, the end-to-end protocol—PUF reconstruction, nonce-bound mutual identification in both directions (the device authenticated the server’s V_B and the server authenticated the device’s V_A in all 30 trials), and AES-256-GCM data exchange—succeeded in every trial, and the corrected verifier V_A matched the single server-registered reference in all 30 reconstructions, confirming that the error-correction layer delivers a stable identity despite per-attempt measurement noise. Second, the reconstruction-attempt pattern is not the gradual, roughly-random bit-error-rate one might expect from thermal noise: every single trial required exactly two attempts—the first always failed the library’s post-mask error-rate threshold (never the hamming-weight check), and the second always succeeded. This perfectly bimodal pattern across 30 independent cold resets suggests a systematic effect tied to the reset/power-up transient of the RTC FAST SRAM region specifically, rather than random analog noise, but the root cause is not established here and is left as an open question; a genuinely random-noise process would not be expected to produce a 30/30 bimodal split this clean, so this finding is reported as-is rather than folded into a single bit-error-rate figure. Third, one of the 30 trials (run 16) measured a handshake latency of 270 ms against 60–80 ms for the other 29; the

Table 19: PUF-based RoT real-hardware bench trial: SMG_NODE_01, $N = 30$ independent hard-reset trials against one fixed enrollment. Attempt-1 diagnostics are from `esp32_puflib`’s own threshold check (hamming-weight must exceed 48.5%; post-mask error rate, over the raw 4096-byte/32768-bit RTC-SRAM region, must stay below 0.15%).

Metric	Value ($N = 30$)	Notes
End-to-end success rate	30/30 (100%)	Mutual identification (P2) + AES-256-GCM demo message exchange (P3), confirmed on both device and server logs
Reconstruction attempts per trial	2/2 in every trial (0/30 succeeded on attempt 1)	Perfectly bimodal across all 30 trials; see discussion below
Attempt-1 hamming-weight	49.949 ± 0.108 % (range 49.673–50.174 %)	Threshold > 48.5 %; passed in all 30 attempts
Attempt-1 post-mask error rate	0.444 ± 0.029 % (range 0.397–0.500 %)	Threshold < 0.15 %; failed in all 30 first attempts, passed on every second attempt
Attempt-1 error bits (of 32768 raw bits)	145.6 ± 9.5 (range 130–164)	Measured before stable-bit masking
Handshake latency (TCP connect $\rightarrow$ session established)	80.7 ± 36.2 ms (range 60–270)	P2 nonce-bound mutual identification; one trial (run 16) measured 270 ms, all other 29 fell in 60–80 ms (median 70 ms); see discussion below
WiFi association latency	3547.7 ± 10.7 ms (range 3530–3570)	ESP-IDF stack retried the first association attempt in nearly every trial; a WiFi-stack characteristic, not a RoT protocol property
Final verifier (V_A) reproducibility	Consistent with a single value in 30/30 trials	All 30 trials authenticated against the one server-registered reference verifier (cryptographic proof match via the mutual-authentication challenge); the current firmware no longer logs the raw corrected value to serial (removed for production hygiene after an earlier internal round), so this is confirmed indirectly through successful authentication rather than by direct hex comparison across trials

gap falls entirely between the server authenticating the device’s proof and the device receiving the session acknowledgment, i.e. on the server round-trip rather than the PUF/cryptographic computation, consistent with—but not confirming—the server-side contention hypothesis already noted for the P4 measurements in the CSQ3 answer above. All latencies in Table 19 are read from the ESP-IDF boot log’s millisecond timestamp, which is itself quantized to a 10 ms step (every value recorded across all 30 trials is a multiple of 10); the reported means and standard deviations should be read at that resolution, not as sub-10 ms-precise measurements, and the 100.0 ± 3.71 ms CSQ3 PSK-era figure and the 80.7 ± 36.2 ms figure here were obtained under different network conditions, firmware, and protocol message counts, so the difference between them should not be read as a protocol-efficiency comparison. The complete raw serial logs for all 30 trials and the parsed summary dataset are published in the companion artifact repository at `case_study_SMG/Implementation/smg_puf_rot_node/bench_results_20261005/` (Materials Availability). An earlier, superseded bench round exists but is not published alongside it: a firmware debug print active at the time (removed before this round) recorded the node’s session verifier in plaintext in that round’s raw logs, and since that value remained the one pinned on the live device, publishing those specific files was judged to leak a live secret rather than a historical one; its existence and discard rationale are documented in `ERRATA.md` (item PR-m2) without reproducing the leaking files.

6.5. CSQ4: Threat Coverage

Tables 20 and 21 map each OWASP IoT Top 10 item to the methodology stage(s) mandating the countermeasure, the protocol phase(s) that provide the implementation-level evidence (cross-referenced with the phase analysis in Section 5.6 and Table 13), and the specific SMG mechanism.

Answer to CSQ4: Unweighted OWASP IoT Top 10 coverage of 7/10 items fully addressed and 3/10 partially addressed. Six of the seven fully-covered items (I1–I3, I6–I7, I9) have direct evidence from the protocol phases P1–P3; the remaining fully-covered item (I5: component vetting) is addressed by Stage 4 policy artifacts, independent of the protocol. The three partial items are I4 (secure update: policy defined at Stage 7.2.2 but signed-OTA implementation deferred to ITER 2), I8 (device management: server-side session logging without a cryptographically verifiable chain, a deliberate design trade-off from dropping the earlier hash-chain audit mechanism—see Limitations), and I10 (physical hardening: inherently outside the scope of

Table 20: OWASP IoT Top 10 coverage analysis — Part 1 (I1–I5). Stage = methodology process mandating the countermeasure. Phase(s) = protocol phase(s) providing implementation evidence (see Table 13); — = addressed by policy/deployment, not protocol.

#	OWASP IoT Risk	Stage(s)	Phase(s)	Evidence in SMG	Coverage
I1	Weak/hardcoded passwords	2.2.1	P1, P2	No password or key exists on the device at all: identity is reconstructed at boot from a PUF (<code>esp32_puflib</code>) using only non-secret mask/ECC helper data in NVS; WiFi pass in NVS; P2 derives the session commitment without transmitting the physical response; confirmed working end-to-end in 30/30 real trials on SMG_NODE_01 (Section 6.4)	Full
I2	Insecure network services	2.2.4	P2, P3	P2 establishes mutual identification and a session key before any data is accepted; P3 encrypts every message with per-message random 12-byte nonce (AES-256-GCM); no plaintext channel exists	Full
I3	Insecure ecosystem interfaces	3.2.1	P2	P2 nonce-bound mutual identification, both parties verified before any session key is derived; entity-type-enforced access control on all API endpoints; trust score < 0.30 disconnects anomalous nodes	Full
I4	Lack of secure update	7.2.2	—	OTA update policy (Stage 7.2.2): admin authentication + firmware signature required; implementation scoped to ITER 2 (policy artifact only, no signed-OTA deployment at TRL 3)	Partial
I5	Insecure/outdated components	4.2	— ³⁹	ESP-IDF v5.4.2 pinned (Stage 4.2 version-pinning policy; version per the flashed firmware build log); no deprecated cryptographic primitives (SHA-256 hash-reconstruction challenge	Full

Table 21: OWASP IoT Top 10 coverage analysis (continued from Table 20) — Part 2 (I6–I10).

#	OWASP IoT Risk	Stage(s)	Phase(s)	Evidence in SMG	Coverage
I6	Insufficient privacy	1.4.3 + 2.2.4	P3	Data sovereignty policy (Stage 1.4.3): self-hosted RPi4; no PII; sensor data only; P3 encrypts all telemetry in transit with authenticated encryption	Full
I7	Insecure data transfer/storage	3.1.1 + 2.2.4	P3	P3 provides AES-256-GCM authenticated encryption per message; no raw secret at rest on the device (P1/P2, PUF-derived); only non-secret mask/ECC helper data and no plaintext credentials in NVS	Full
I8	Lack of device management	7.1	—	Stage 7.1 server monitors heap, send rate, and session status from server-side logs (node_id, timestamps); without a cryptographic chain, this is an operational log rather than an independently verifiable activity audit	Partial
I9	Insecure default settings	2.2.3 + 6.1.3	P1	P1 enrollment is a boot prerequisite: the firmware cannot reach WiFi or server communication until enrollment data (mask/ECC) exists in NVS, and enrollment derives the identity proof from the physical PUF response of the specific chip rather than from any stored or default value, so no two chips can share a cloned identity. Bring-up	Full

a software development methodology). The coverage score is an unweighted count of addressed categories, not a severity-weighted risk-reduction metric (see B12 discussion, Section 6.6).

6.6. Expert Validation

A structured questionnaire (14 Likert items, B1–B14, 5-point scale from 1 = strongly disagree to 5 = strongly agree, plus 3 open questions) was distributed to domain experts to supplement the case study evidence with independent professional judgment. Fifteen evaluators completed the questionnaire; fourteen consented to their responses being used as validation evidence (one declined consent and was excluded prior to analysis), exceeding the upper bound of the planned 6–10 evaluator target. Six requested anonymity and are referenced anonymously below. Evaluators held primary expertise in cybersecurity, IoT systems design/development, smart-grid/energy management, software engineering, or embedded systems, with 1–15 years of professional or research experience and academic backgrounds ranging from B.Sc./B.Eng. to Ph.D., from institutions in Mexico, Spain, Portugal, and Canada. Eight of the fourteen are affiliated with institutions unconnected to the author team (CINVESTAV, ESCOM-IPN, a national Instituto Tecnológico, ECOSUR, Universidad Iberoamericana, Universidade de Aveiro, University of New Brunswick), making the panel majority-independent; the remaining six are affiliated with the author institutions (INAOE, home institution; CEIT-BRTA, co-author affiliation).

Quantitative results: the overall mean across all 196 Likert responses (14 evaluators $\times$ 14 items) was 3.98/5, with item means ranging from 3.57 (B12: OWASP coverage sufficiency) and 3.71 (B1: overall structure sufficiency) to 4.43 (B10: hash-chain integrity guarantees) and 4.36 (B3: stage-by-stage artifact verifiability; B14: more systematic integration than existing methodologies). No item fell below the questionnaire’s neutral midpoint (3.0) on average, and the highest-rated item (B10) corresponds directly to CSQ3. The independent-evaluator subset ($N=8$) returned an overall mean of 3.89/5, modestly below the affiliated subset (4.11/5) and the full panel; the direction of this gap indicates mild affiliation skew, which the majority-independent composition of the panel mitigates.

Qualitative findings that qualify the results: B12 (OWASP coverage sufficiency) returned the lowest item mean (3.57/5). Two evaluators independently qualified their B12 agreement on the same point: the 9/10 OWASP coverage score (as presented to evaluators) is an unweighted count

Table 22: Expert questionnaire Likert results (N=14; 1 = strongly disagree, 5 = strongly agree).

Item	Statement (abbreviated)	Mean
B1	Four-phase, seven-stage structure gives sufficient guidance without external references	3.71
B2	Organizational phase (Stage 1) adds practical value ahead of architecture/technology decisions	3.79
B3	Stage-by-stage artifact requirements make correct methodology application verifiable	4.36
B4	TRL-guided iterative model with M1–M3 maintenance points is practical	3.86
B5	Entity-based identity model is clearer than role/policy-based approaches	3.86
B6	12-parameter identity model generalizes beyond the SMG case study	3.86
B7	RoT-as-process abstraction accommodates heterogeneous device capabilities	4.29
B8	Dynamic RoT execution (Process 6.1.3) is a verifiable identity mechanism	4.00
B9	Six-phase authentication/data-exchange protocol (P1–P6) is logically complete	3.86
B10	Hash-chain mechanism (P4/P6) gives meaningful, feasible integrity guarantees	4.43
B11	Technology-independent STRIDE mapping is more useful than technology-specific models	4.07
B12	90% OWASP IoT Top 10 coverage is sufficient security-by-design evidence at TRL 3	3.57
B13	Parameterized, technology-agnostic design applies beyond energy management	3.79
B14	More systematic security integration than existing IoT methodologies	4.36
Overall mean (196 responses)		3.98

Note (B12): item text reflects OWASP coverage as presented to evaluators (9/10 full + 1/10 partial = 90%); subsequent reclassification of I4 (secure update) from Full to Partial, and later of I8 (device management) from Full to Partial following the removal of the hash-chain audit mechanism in the PUF-based redesign, yield the revised 7/10 full + 3/10 partial reported in CSQ4 (Section 6).

Note (B6, B7, B9, B10): these item texts, as presented to evaluators, reference the prior PSK+NVS instantiation (12-parameter identity model; six-phase P1–P6 protocol; hash-chain integrity mechanism). The PUF-based redesign (Section 6.4) supersedes those mechanisms (14 parameters; P1–P3; no hash chain). The panel was not re-surveyed on the redesigned mechanisms, so these means are retained as historical instrument data validating the prior instantiation’s design choices, not the current redesign.

of addressed categories, not a severity-weighted risk-reduction metric, and does not by itself establish that the one partially-addressed item (I10, physical hardening) is the least critical omission; subsequent review (Section 6, CSQ4 answer) reclassified I4 (secure update) from full to partial on the same unweighted-count basis, and a later reclassification of I8 (device management) from full to partial followed the removal of the hash-chain audit mechanism in the PUF-based redesign, together yielding a revised 7/10 full + 3/10 partial; this clarification is reflected in the CSQ4 discussion (Section 7). B1 (structure sufficiency) returned the second-lowest mean (3.71/5): three evaluators scored it 2/5, with comments indicating that the methodology specifies *what* each stage must produce but not *how*—per-artifact templates, worked examples, or acceptance criteria would close this gap—a recommendation echoed by several open-question (C3) responses (stepped adoption guidance, per-artifact templates, additional use cases). One evaluator qualified B6 by noting that the parameter count is an artifact of the SMG’s four entity types rather than a fixed model limit (12 parameters under the prior PSK+NVS instantiation presented to evaluators; 14 after the PUF-based redesign, Table 12), but cautioned that generalization to other entity types remains untested. Two evaluators gave B13 (cross-domain generalizability) dissenting scores (2/5 and 1/5), stating that a high degree of generalizability is difficult to achieve in IoT and should be treated as a hypothesis rather than a demonstrated property. These responses are reflected in the External Validity threat and Limitations below. One evaluator’s comments referenced preliminary bench-trial figures from the condensed reference document distributed with the questionnaire (finalized June 24, 2026, before the final multi-run trials); those figures are more conservative than the final measurements reported in Table 17, so that evaluator’s positive assessment understates the methodology’s demonstrated performance. One evaluator questioned whether case study evidence produced by the same team that authored the methodology is sufficient without an independently-managed development team; the SMG implementation was carried out by multiple co-authors in distinct roles (see Author Contributions), which partially, but not fully, addresses this internal-validity concern. Two evaluators, whose primary expertise is software-engineering methodology design and general IoT systems development respectively (not cybersecurity specifically), noted for several protocol- and threat-model-specific items that those items fell outside their domain of expertise; their ratings are retained in the aggregate above for completeness but should be weighted accordingly when interpret-

ing individual item results. The panel’s majority-independent composition and the corresponding subset means are reported above; the residual affiliation skew, the convenience-sampling constraint on recruitment, and the preliminary-data caveat are acknowledged as limitations below.

7. Discussion

This work addresses two barriers to secure IoT ecosystem development: the security activation gap at the operational level, and the absence of a structured development methodology that integrates cybersecurity from the earliest organizational stage. The results of the confirmatory case study and expert validation are discussed below in relation to each research question.

CSQ1 — Methodology guidance: The stage-by-stage traceability (Tables 10 and 11) demonstrates that the methodology produces actionable decision points at every stage. The organizational phase (Stage 1) carries the highest leverage: classifying the SMG as critical infrastructure (Process 1.4.1) directly constrained the authentication scheme (requiring mutual cryptographic authentication), the storage approach (requiring secure hardware isolation), and the access control granularity (requiring node, user, and admin distinction). Absent an explicit organizational stage, these constraints might not have been systematically propagated to architectural and implementation decisions.

CSQ2 — Identity model: The formal separation of identity into P_{pub} , P_{priv} , and P_{auth} subsets yielded a technology-agnostic security specification that translated without modification to ESP32 NVS storage for public helper data and to SHA-256 hashing (via mbedtls) for the private, non-persistent PUF response. The 14-parameter model produced zero ambiguity across 4 entity types, confirming the model’s expressiveness for heterogeneous IoT ecosystems, including resource-constrained devices.

CSQ3 — Root of Trust verifiability: The P2+P3 mutual authentication handshake completed in 56.7 ms (NODE_02, avg 2 runs) and 57.6 ms (NODE_01), well within the firmware-guaranteed 1 s maximum inter-send interval. P4 per-message round-trip latency averaged 30.3 ± 3.3 ms over 196 pooled bench messages from NODE_02 and 33.0 ± 4.1 ms over 120 messages from NODE_01 (Table 17). Notably, the P2+P3 handshake latency was dominated by WiFi round-trip time rather than cryptographic computation ($\text{HKDF} + \text{AES-256-GCM} < 1$ ms). The root cause of P4 per-message latency is less certain: a companion PC-client (CPython) bench trial against

the same server, predating the ESP32 firmware fixes, attributed its P4 latency to server-side scheduling artifacts—the primary agent’s background EMS and status loops acquiring a data-store lock that serializes P4 handling (uncontended baseline $\approx 1.7\text{ms}$ on that client). However, this diagnosis has not been confirmed on the ESP32 platform reported here: the ESP32 P4 standard deviations (3.3/4.1ms) are far tighter than the bimodal contended/uncontended split the lock-contention mechanism predicts, and no per-message ESP32 timestamps were retained to test for that pattern directly. Whether WiFi/TCP stack overhead, MicroPython cryptographic computation, server-side contention, or some combination dominates the ESP32’s $\sim 30\text{ms}$ P4 latency therefore remains an open question (see Limitations); this uncertainty concerns only the *causal explanation* of the latency, not the validity of the measurement itself or of the RoT-verifiability evidence, which rests on successful mutual authentication and chain integrity rather than on which mechanism dominates per-message timing. Hash-chain integrity reached 100% across all verified messages on both nodes with zero security errors, confirming continuous session integrity verification; send success rate was 100%. Heap drift of $-7\,632\text{B}/100$ loops (NODE_02) and $-7\,344\text{B}/100$ loops (NODE_01) reflects chain state accumulation within the bench-trial windows (see Limitations for the sustained-operation implications).

CSQ4 — Threat coverage: The unweighted OWASP IoT Top 10 coverage (7/10 items fully addressed; 3/10 partially addressed: I4 secure-update policy only, I8 device management without a cryptographically verifiable chain, I10 physical hardening) indicates that the methodology’s security-by-design approach systematically addresses known IoT threat categories. The STRIDE analysis operates at two levels: the methodology-level mapping (Tables 5 and 6, Section 4.7) shows that every STRIDE threat is addressed by a technology-agnostic mechanism mandated by at least one methodology stage; the protocol-level mapping (Tables 13–14, Section 5.6) traces each mechanism to the specific phase that implements it in the SMG. All six STRIDE categories are addressed, and four are covered by multiple independent phases. The three partial OWASP items (I4, I8, and I10) reflect two deliberate design trade-offs (I4, I8—see Limitations) and one inherent scope boundary (I10) of a software development methodology; the coverage figure is an unweighted count, not a severity-weighted risk-reduction metric.

Maintenance cycles (historical, prior PSK+NVS/MicroPython scheme): Three maintenance moments were triggered during Stages 5–6 of the earlier implementation (one M1 firing for technology re-evaluation,

two M2 firings for code revision), confirming that the methodology’s M1 and M2 maintenance pathways operate as specified at TRL 3. All three issues were caught during the systematic implementation review prescribed by the methodology, before deployment. Absent the methodology’s structured review requirements at each stage transition, these bugs would likely have been discovered only under operational conditions. The M3 (architectural redesign) mechanism was not triggered by any TRL 3 incident and therefore remains empirically unvalidated; it is available for the operation-phase TRL transition.

7.1. Limitations

- **Single topology:** Only centralized topology was implemented at TRL 3. Decentralized and mesh topologies supported by Stage 2.1.4 remain to be validated empirically.
- **TRL 3 scope:** The proof of concept operates in a controlled laboratory environment. Environmental variation, scalability beyond 2 nodes, and production-grade hardening (JTAG disable, secure boot) are deferred to ITER 2.
- **Primary-agent power dependency:** In the TRL 3 bench configuration the RPi4 primary agent is powered from the same DC bus it coordinates (via a DC/AC inverter); SSR duty transients and node resets cause primary-agent brownouts that terminated several bench trials early. This coupling constrains sustained concurrent operation and is an artifact of the laboratory power arrangement rather than the methodology; ITER 2 will use an independent primary-agent supply. The bench-trial windows reported in Table 17 (NODE_01: 120 sends; NODE_02: 196 sends pooled from 108 + 88; each ~30–40 s plus 20 warm-up) partly reflect this constraint.
- **Frozen-module firmware constraint (historical, prior MicroPython scheme):** Under the earlier PSK+NVS/MicroPython instantiation, ESP32 security modules had to be frozen into the firmware image (not loaded from the flash filesystem at runtime) to preserve heap stability during WiFi + crypto coexistence; runtime filesystem import of the crypto modules corrupted lwIP socket state, constraining the OTA update path (Stage 7.2.2) to full-image replacement. This constraint is specific to MicroPython’s module-loading model and does not apply to the C/ESP-IDF

firmware, where all code is statically linked into a single image regardless; it is retained here as a documented finding from the prior instantiation, not a limitation of the current design.

- **Forward secrecy (symmetric RoT instantiation):** The PUF-based hash-reconstruction scheme instantiated for the constrained ESP32 nodes does not provide forward secrecy: $Key_{ab} = \text{SHA-256}(V_A \parallel V_B \parallel N_A \parallel N_B)$ combines long-term verifiers with nonces exchanged in the clear, so an attacker who later compromises V_B (server store) or physically extracts V_A (device PUF response), and has recorded a past handshake’s nonces, can recompute that session’s key. This is a property of the symmetric Stage 2.2.3 RoT variant at TRL 3; the asymmetric RoT variant or an ephemeral-DH extension is the ITER 2 remedy.
- **Central-server verifier storage (reformulated under the PUF-based redesign):** Under the PUF-based scheme, the RPi4 stores only a per-node hash verifier ($V_A = \text{SHA-256}(\text{PUF}_A)$), not a raw secret; server compromise no longer directly discloses a value that recreates a node’s physical identity, in contrast to the earlier PSK design’s `node_key` storage. This mitigates, but does not eliminate, the single-point-of-failure risk: if the PUF response’s entropy is insufficient, offline brute-force or precomputation attacks against the stolen verifier remain possible, and this depends on the actual (not merely designed) entropy of the PUF response on the deployed hardware. An enrollment run on one of the SMG’s physical ESP32 nodes measured an 418-byte (~ 3344 -bit) stable response — somewhat above the ~ 380 – 400 -byte figure the library’s own design notes estimate for its reference hardware, a chip-to-chip variation discussed below — and, at that raw length, comfortably sufficient to seed a 256-bit key derivation; this bounds the response’s raw length but is not itself a formal min-entropy measurement, which [TODO: remains pending a dedicated min-entropy assessment (e.g., NIST SP 800-90B)]. ITER 2 will evaluate per-node verifier wrapping or HSM-backed server storage as defense in depth regardless.
- **PUF library build and bench status (validated on one of two nodes):** `esp32_puflib` (Stanicek, 2022) is a native C/ESP-IDF component and is linked directly into the node’s C/ESP-IDF firmware (`smg_puf_rot_node`), without a language-runtime wrapper; this removes the cross-runtime integration risk of the prior MicroPython-based plan. The identity/Root-of-Trust/secure-communication firmware and its RPi4 counterpart (`rot_challenge_service.py`)

build cleanly, and the firmware has since been flashed to, enrolled on, and bench-validated end-to-end against one of the SMG’s two physical ESP32 units: $N = 30$ real hard-reset trials against the real RPi4 server reached 30/30 end-to-end success (Section 6.4). SMG_NODE_02 has not yet been flashed or enrolled under this scheme, so two-node validation, and any claim of consistency across physically distinct chips, remains open.

- **Sensor/EMS modules not yet re-reported (C/ESP-IDF firmware scope):** The new `smg_puf_rot_node` C/ESP-IDF firmware currently implements only the identity (PUF), Root of Trust, and secure-communication layers; the fuzzy Energy Management System and sensor-acquisition logic previously implemented in MicroPython (`ems_module.py`, `sensor_module.py`) have not yet been re-reported to C/ESP-IDF. A full end-to-end SMG deployment under the new architecture requires this additional porting work, distinct from and in addition to the hardware bench validation above.
- **No cryptographic session-chain audit (deliberate design simplification):** An earlier instantiation of this methodology’s Root of Trust used a per-message hash chain (M_{num} , PRV_{bl}) to provide a retrospectively-verifiable, third-party-auditable record of session integrity, directly supporting Repudiation coverage and OWASP I8 (device management). The PUF-based redesign drops this mechanism in favor of a smaller on-device footprint: message-level integrity is still provided by the AES-256-GCM authentication tag (P3), but there is no cryptographic chain binding messages within a session, and Repudiation/I8 coverage is now evidentiary (server-side session logs) rather than independently verifiable (Table 5, Table 21). Reintroducing a lightweight, chain-free sequencing mechanism (e.g., a monotonic per-session counter bound into the GCM associated data) without the full chain-audit overhead is a candidate ITER 2 refinement if reviewers or subsequent evaluation identify this trade-off as unacceptable.
- **PUF reliability on the deployed hardware (reconstruction reliability measured on one node; thermal/second-node characterization pending):** `esp32_puflib`’s enrollment procedure targets a design bit-flip probability of 0.001–0.005 via a stable-bit mask and 8×-repetition-code error correction, yielding an estimated ~ 380 –400-byte stable response in the library’s own design notes for its reference hardware at that setting.

Five independent enrollment runs on SMG_NODE_01 (ESP32-D0WD-V3 rev 3.1) produced stable responses of 418, 421, 420, 420, and 420 bytes — consistently reproducible across separate enrollments, but somewhat above that ~ 380 – 400 -byte figure, a plausible chip-to-chip or measurement-setup variation rather than a conformance failure, since the library documents its figure as an estimate from its own test hardware rather than a hard specification. Reconstruction reliability against one fixed enrollment was then measured over two independent $N = 30$ hard-reset bench rounds (Table 19, Section 6.4): in both rounds, every trial required exactly two reconstruction attempts (never one, never three or more), with the first attempt always failing the library’s post-mask error-rate threshold (most recently $0.444 \pm 0.029\%$, threshold $< 0.15\%$) while always passing the hamming-weight check (most recently $49.949 \pm 0.108\%$, threshold $> 48.5\%$), and the second attempt always succeeding, reproducing a verifier consistent with the single server-registered reference in all 30 trials of the most recent round. This perfectly bimodal pattern is itself an open question (Section 6.4 discusses a plausible reset-transient explanation that is not confirmed) and should not be read as a conventional, gradually-distributed bit-error-rate. What remains unmeasured is (a) sensitivity to temperature and supply-voltage variation, and (b) whether the same two-attempt pattern and response length hold on the second node (SMG_NODE_02) and across independent enrollments rather than one fixed enrollment: [TODO: measured temperature/voltage sensitivity and cross-enrollment, c remains pending before the scheme’s reliability claim extends beyond SMG_NODE_01 under laboratory-ambient conditions.

- **Heap drift and sustained-operation ceiling:** Measured per-loop heap drift ($-7\,632\text{ B}/100$ loops on NODE_02; $-7\,344\text{ B}/100$ loops on NODE_01) against a $\sim 116\text{ KB}$ session heap implies that, if the drift rate held constant, heap would exhaust after approximately 6 minutes of continuous operation. The bench trials (~ 30 – 40 s , well inside this window) were too short to expose or confirm this extrapolated trend. A longer diagnostic run to isolate whether the drift is the documented $32\text{ B}/\text{loop}$ chain-state growth, garbage-collector scheduling behavior, or an unaccounted leak was not performed: it requires the same sustained primary-agent uptime that the power dependency described above (RPi4 brownouts on SSR duty transients) currently precludes at TRL 3. This uncertainty bounds only the firmware’s demonstrated *continuous*-operation duration; it does not af-

fect the validity of the bench-trial measurements reported in Table 17, all of which were taken well within the pre-exhaustion window. Root-cause analysis and a GC-tuning fix, using the methodology’s own Stage 7.1 monitoring (heap free) as the operational detection mechanism, are flagged for resolution before advancing to ITER 2.

- P4 latency root cause (ESP32 platform):** The decomposition of the ESP32’s ~30 ms P4 per-message latency among WiFi/TCP round-trip overhead, MicroPython cryptographic computation, and server-side scheduling contention is not established for the platform actually reported here (see CSQ3 discussion above). A lock-contention mechanism identified in a companion PC-client (CPython) bench trial against the same server—run before the ESP32 firmware fixes, on different hardware and client software—is a plausible contributor, but is not confirmed on the ESP32: the ESP32 P4 standard deviations (3.3/4.1 ms) are far tighter than the bimodal contended/uncontended split that mechanism predicts, and per-message ESP32 timestamps needed to test for it directly were not retained, nor was a dedicated diagnostic run performed, for the same primary-agent power-dependency reason given above. This affects only the causal explanation of the latency value, not its validity as a measurement: the P4 averages and standard deviations in Table 17 are real, reproducible bench data, and CSQ3’s Root-of-Trust-verifiability claim rests on successful mutual authentication and 100% chain integrity across all measured messages, neither of which depends on which mechanism dominates per-message timing. Capturing per-message ESP32 timestamps in a future bench trial is the direct resolution path and is deferred to ITER 2 alongside the heap-drift investigation above.
- Single application domain:** Energy management is the only domain tested. The expert questionnaire (N=14, Section 6.6) supports the technology-agnostic design intent (B13 mean 3.79/5) but also recorded two dissenting scores on this same item (2/5 and 1/5), with the dissenting evaluators characterizing cross-domain generalizability as an unvalidated hypothesis; empirical validation in additional domains remains future work.
- Physical hardening:** I10 (physical hardening) and hardware-based Root of Trust (e.g., ATECC608 secure element) are not addressed at TRL 3 but are specified as ITER 2 requirements.

- **Identity lifecycle (methodology gap):** The methodology as presented does not specify a complete identity lifecycle process (parameter rotation, revocation, recovery after compromise without full re-provisioning, ownership transfer, and decommissioning). This is a gap in the methodology itself—not solely a TRL 3 scope limitation—since recovery from a compromised node currently requires re-running the full Root of Trust. An expert evaluator (Section 6.6) identified this as the single addition with the greatest potential impact on practical applicability; it is the highest-priority extension for the next methodology revision and is deferred to ITER 2.
- **Expert validation:** Fourteen evaluators completed the questionnaire (14 Likert items, 3 open questions), exceeding the planned 6–10 target and yielding an overall agreement mean of 3.98/5 (Section 6.6). Eight of the fourteen are affiliated with institutions unconnected to the author team, making the panel majority-independent; recruitment nonetheless remained convenience-sampled. The independent subset (mean 3.89/5) rated modestly below the affiliated subset (4.11/5), indicating mild affiliation skew that the panel composition mitigates. The condensed reference document distributed to all evaluators (June 24, 2026) contained preliminary bench trial data; final performance measurements reported in Table 17 reflect updated trials conducted after distribution and are more favorable (e.g., handshake 56.7 ms vs. preliminary 257.6 ms). One additional respondent declined consent and was excluded prior to analysis.

7.2. Threats to Validity

8. Conclusions

This work has presented a security-by-design methodology for centralized IoT ecosystems that addresses the security activation gap by integrating cybersecurity systematically from the earliest organizational stage. The methodology’s four phases and seven stages produce documented, traceable artifacts; the entity-centric identity model provides technology-agnostic security specifications applicable to heterogeneous devices; and the dynamic Root of Trust process establishes verifiable cryptographic identity before any operational communication begins.

Empirical validation through a confirmatory case study (Yin, 2014) applying the methodology to a DC Smart Microgrid IoT ecosystem yields four findings. First, all seven stages produced complete, traceable outputs

Table 23: Threats to validity.

Type	Specific Threat	Mitigation
Internal validity	SMG decisions may reflect author preference rather than methodology guidance	Stage-by-stage traceability (Tables 10 and 11) documents explicit methodology process → decision chain for each output
External validity	Single domain (energy) + single topology (centralized) limits generalizability	Methodology is technology-agnostic by design; expert questionnaire (N=14, mean 3.98/5, Section 6.6) supports this design intent, though three evaluators noted that generalization beyond the validated domain and entity set remains an untested hypothesis
Construct validity	TRL 3 does not demonstrate production readiness	TRL 3 is the stated objective of ITER 1; the methodology explicitly supports iterative maturation toward TRL 5+
Reliability	Physical prototype measurements subject to environmental variation	Prior PSK+NVS scheme: two independent bench trials on SMG_NODE_02 (196 pooled bench messages) and one recovery trial on SMG_NODE_01 (120 messages); P4 std 3.3/4.1 ms (N2/N1); heap drift −7 632/−7 344 B/100 loops; chain integrity 100% on both nodes. PUF-based redesign: an additional single-node bench ($N = 30$ hard-reset trials on SMG_NODE_01, Section 6.4); its single-node scope and ambient laboratory conditions are disclosed limitations of that stream

with zero external guidance required, confirming methodology sufficiency (CSQ1). Second, the 14-parameter identity model across four entity types produced zero ambiguity and translated directly to implementation artifacts (CSQ2). Third, the RoT process, instantiated via a PUF-based identity mechanism (`esp32_puflib` (Stanicek, 2022)) that reconstructs each node’s identity from non-persistent physical characteristics rather than a stored key, established mutual authentication with 56.7 ms total P2+P3 handshake latency (NODE_02) and 57.6 ms (NODE_01)—handshake latency dominated by WiFi RTT—with per-message P4 latency of 30.3 ± 3.3 ms (NODE_02, 196 messages) and 33.0 ± 4.1 ms (NODE_01, 120 messages), 100% chain integrity, and zero security errors on both nodes, confirming deployability on resource-constrained ESP32 hardware (CSQ3). These figures describe the case study’s original PSK+NVS instantiation; the subsequent PUF-based redesign was independently validated end-to-end on real hardware for one of the two nodes (SMG_NODE_01, $N = 30$ hard-reset trials against a real WiFi access point and primary-agent server): 30/30 end-to-end success and a mean mutual-authentication handshake latency of 80.7 ± 36.2 ms, with PUF-response reconstruction requiring exactly two attempts in every trial, a perfectly bimodal pattern left as an open question rather than attributed to a confirmed cause (Section 6.4); validating SMG_NODE_02 under this scheme remains future work. Fourth, the methodology achieved unweighted OWASP IoT Top 10 coverage of 7/10 items fully addressed and 3/10 partially addressed (I4 secure-update policy only; I8 device management, an explicit trade-off from dropping the earlier hash-chain audit mechanism; I10 physical hardening outside software scope) (CSQ4).

The methodology’s parameterized, technology-agnostic structure is designed to support application across diverse IoT domains; cross-domain generalization is a design intent supported by expert opinion (B13 mean 3.79/5, with dissenting 2/5 and 1/5 ratings characterizing it as an unvalidated hypothesis) rather than a demonstrated property, and empirical validation in additional domains remains future work. Future work will extend validation to: ITER 2 (TRL 5+ with hardware security elements and signed-OTA implementation), cross-domain deployment (healthcare, industrial automation), an expanded expert evaluator pool with improved institutional independence, formal cryptographic protocol verification of the SMG’s PUF-based hash-reconstruction challenge scheme, resolution of the residual heap drift and the identity-lifecycle gap (rotation, revocation, recovery), evaluation of per-node verifier wrapping or HSM-backed server storage to re-

duce central-server single-point-of-failure risk, and a lightweight chain-free sequencing mechanism to recover stronger Repudiation/I8 coverage without reintroducing the discarded hash-chain audit’s on-device footprint.

Declarations

Materials Availability

The ESP32 firmware suite (`smg_control_node/`, original PSK+NVS/MicroPython instantiation; `smg_puf_rot_node/`, PUF-based redesign) and primary agent software (`smg_primary_server/`, covering both instantiations) are available under Apache 2.0 license in the companion code repository: <https://github.com/RazielCS/smg-secure-microgrid>. The complete research artifacts — manuscript source, case-study documentation, raw bench-trial measurements for both instantiations (canonical CSVs, serial logs, packet captures; the PUF redesign’s $N = 30$ bench data under `case_study_SMG/Implementation/smg_puf_rot_node/`), the frozen firmware image used on the original bench, the expert-validation instruments, and this errata log — are available in the companion artifact repository: <https://github.com/RazielCS/smg-secure-microgrid-paper-artifacts>. Device- and server-side secrets (the real PUF verifiers, NVS images, and WiFi credentials) are excluded from both repositories and replaced with sanitized `.example.json` templates and placeholder values; see each repository’s README for the exact exclusion list.

Competing Interests

The authors declare no competing financial interests or personal relationships that could have appeared to influence this work.

Funding

R. Campos-Sanchez thanks SECIHTI for scholarship support (CVU 623274).

Acknowledgements

The authors thank the following experts for completing the validation questionnaire (Section 6.6) and consenting to attribution: Jose de Jesus Zapata Lara (CINVESTAV); Jorge Andres Larracochea, PhD (Asociacion Centro Tecnologico Ceit); Claudia García-Blanquel, PhD (ESCOM-IPN); Edwin Pacheco, MSc student in Security Sciences and Technologies (INAOE); Agustin Alberto Ahumada Ramirez, MSc student in Security Sciences and

Technologies (INAOE); Yair Romero Lopez, M.Eng. (INAOE); Alfonso Martínez-Cruz, PhD (SECIHTI-INAOE); and Eder Toledo, MSc (ECOSUR). Six additional evaluators contributed responses under conditions of anonymity and are not named here per their request. One further respondent declined consent to the use of their responses and was excluded prior to analysis.

Author Contributions

R. Campos-Sanchez: Conceptualization, investigation, formal analysis, writing — original draft, implementation. **L. Hernandez-Martinez:** Supervision, validation, writing — review & editing. **C. Feregrino-Uribe:** Supervision, validation, writing — review & editing. **A. Penuelas-Angulo:** Conceptualization, formal analysis, writing — original draft. **D. Cruz-Aguilar:** Conceptualization, validation, implementation, writing — review & editing.

Appendix A Methodology Parameterized Specification: SMG Case Study

This appendix provides the full parameterized specification of all four phases for the SMG case study, complementing the condensed traceability in Tables 10 and 11. Each row in Tables 24–27 corresponds to a methodology parameter defined in the methodology’s process tables. Codes with an additional digit (e.g., 4.1.2.1) refine a stage-level process (as cited in the main text) into its implementation-specific components; the parent code is the methodology-level process that those refinements collectively satisfy.

References

- Abraham, D., Houmb, S.H., Erdodi, L., 2025. Cyber-attacks on energy infrastructure—a literature overview and perspectives on the current situation. *Applied Sciences* 15, 9233.
- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., Ayyash, M., 2015. Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials* 17, 2347–2376.
- Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J.A., Invernizzi, L., Kallitsis, M.,

Table 24: Phase A, Stage 1: Organizational aspects — SMG application.

Process ID	Parameter	SMG Value
1.1	Problem statement	Manage distributed DC renewable energy generation, storage, and consumption across 2 co-located laboratory nodes; provide global coordination and real-time monitoring without centralized national grid
1.1.1	Cost-benefit justification	Energy losses (imbalanced generation/demand) eliminated; TRL 3 budget: 2× ESP32 NodeMCU + RPi4 + commercial solar adapters + AGM batteries + IBFIA SSRs
1.1.2	Entities involved	5: IoT nodes (2× ESP32), IoT server (RPi4), users (operators), administrators (researcher), DC bus (physical)
1.3.1	Development iteration (TRL)	ITER 1 target: TRL 3 (proof of concept with physical elements, controlled laboratory)
1.4.1	Criticality classification	Critical infrastructure — electrical energy; high-security tier selected (Stage 1.4.1)
1.4.2	Technological sovereignty	No cloud services; all computation on ESP32 (local) and RPi4 (global); open-source software only
1.4.3	Data sovereignty	Self-hosted RPi4 server; no third-party cloud; all sensor data stored locally
1.4.4	Access control policy	Token-based; nodes: POST sensor data only; users: GET monitoring; admins: full configuration and entity management
1.4.5	Communication policy	Mandatory application-layer encryption (AES-256-GCM); TCP over WiFi 802.11 b/g/n; no plaintext data channel
1.4.6	Applicable guidelines	Secure-by-Design principles (UK Government Security, 2024); NIST IoT security (Fagan et al., 2021); OWASP IoT Top 10

Table 25: Phase B, Stages 2–3: Architecture and service design — SMG application.

Process ID	Parameter	SMG Value
2.1.4	Communication topology	Centralized (hub-and-spoke): both nodes communicate exclusively with primary server; no node-to-node direct communication
2.1.5	Communication infrastructure	WiFi 802.11 b/g/n; ESP32 built-in; TCP sockets; RPi4 configured as WiFi AP (SSID: SMG_Primary; hostapd configuration, 192.168.4.0/24, for the PSK-era bench; NetworkManager hotspot, 10.42.0.0/24 with gateway 10.42.0.1, for the PUF-based bench, Section 6.4)
2.1.7	Processing topology	Fog: local 9-rule fuzzy EMS on each ESP32; global 27-rule security-aware fuzzy EMS on RPi4; commercial solar adapter handles droop and battery-charging voltage regulation autonomously
2.1.8	Entity hierarchy	IoT Nodes < Users < Administrators
2.2.2	Authentication mechanism	Nonce-bound SHA-256 hash-reconstruction challenge (PUF-derived verifier) for IoT nodes; SHA-256-based hash for users/admins
2.2.3	Root of Trust process	PUF-based scheme (Tables 2–3, <code>esp32_puflib</code>): enrollment via RTC-SRAM measurement (admin workstation) → non-secret mask/ECC helper data to NVS (USB serial) → server verifier registration → per-boot response reconstruction and mutual identification
2.2.4	Secure communications	AES-256-GCM (ESP-IDF <code>mbedtls</code> / <code>esp_aes</code>) with 12-byte random nonces per message, keyed from the PUF-derived session key (Table 3); no message-level cryptographic chain (design trade-off, see Limitations); no TLS certificate management needed on constrained devices
3.1.1	Services defined	4: sensor data ingestion (nodes→server), EMS coordination (server→nodes), authentication API (all entities), monitoring dashboard (server→users)
3.2.1	Access control	RBAC via session token type; API endpoints enforce entity class; nodes POST-only to own data endpoint
3.2.2	Identity storage	ESP32: NVS flash partition (separate from firmware, survives updates, JTAG-disabled for TRL 5+); Server: PostgreSQL-equivalent for SMG + <code>node_registry.json</code> for provisioning records

Table 26: Phase C, Stages 4–5: Implementation technologies and functional elements — SMG application.

Process ID	Parameter	SMG Value
4.1.1	TRL 3 criteria met	Physical nodes operational; mutual authentication functional; PWM energy control active; sensor data logged on primary server; all 8 functional tests passed
4.1.2.1	Node platform	ESP32 + C/ESP-IDF (v5.4.2, per the flashed firmware build log): built-in WiFi, SHA-256 HW acceleration, NVS; <code>esp32_puflib</code> (Stanicek, 2022) linked directly as a native ESP-IDF component (no language-runtime wrapper needed); identity/RoT/communication firmware (<code>smg_puf_rot_node</code>) builds cleanly under ESP-IDF; flashed, enrolled, and bench-validated end-to-end on SMG_NODE_01 ($N = 30$ real hard-reset trials, 30/30 success, Section 6.4) — SMG_NODE_02 bench status still pending
4.1.2.2	Cryptography	SHA-256 hash-reconstruction challenge applied to the PUF response (<code>esp32_puflib</code>) for mutual identification, in place of a stored <code>node_key</code> ; SHA-256 (not HKDF) for session-key derivation ($Key_{ab} = \text{SHA-256}(V_A \ V_B \ N_A \ N_B)$, <code>rot_session.c</code>); AES-256-GCM via ESP-IDF <code>mbedtls/esp_aes</code> (no MicroPython, no third-party <code>aesgcm</code> module)
4.1.3	Communication protocol	TCP over WiFi 802.11 b/g/n; application-layer nonce-based hash-reconstruction challenge protocol (implementation-specific Stage 4 choice, presented as part of this work’s own case study; Section 5.6)
4.2.1	Vulnerability mitigations	ESP-IDF v5.4.2 pinned (per the flashed firmware build log); WiFi credentials in NVS; all AES-GCM nonces and challenge values from <code>esp_fill_random()</code> (ESP-IDF HW RNG); no persistent <code>node_key</code> exists to protect (removed by the PUF-based redesign); JTAG disable deferred to TRL 5+
5.1.1	Primary agent (RPi4)	Python: <code>server.py</code> , <code>auth_service.py</code> , <code>ems_coordinator.py</code> (27-rule security-aware fuzzy), <code>data_store.py</code> , <code>crypto_utils.py</code> (prior PSK-based protocol stack, retained for the historical bench data in Section 6); <code>puf_server.py</code> + <code>rot_challenge_service.py</code> (new PUF-based hash-reconstruction challenge service, separate port, Section 5.6)
5.1.2.1	Node power supply	DC-DC buck converter (12V→5V) per node from local battery; isolates ESP32 from shared DC bus; $\approx 90\%$ efficiency vs. $\approx 27\%$ for LDO at 12 V (avoids ≈ 3.8 W peak dissipation)
5.1.2.2	Identity storage	ESP32 NVS: <code>node_id</code> , <code>node_uuid</code> , PUF stable-bit mask, PUF ECC helper data <code>wifi_ssid</code> <code>wifi_pass</code> — none

Table 27: Phase D, Stages 6–7: Deployment, operation, and maintenance — SMG application.

Process ID	Parameter	SMG Value
6.1.3	RoT execution	PUF-based scheme, executed and validated on SMG_NODE_01 (1/2 nodes; SMG_NODE_02 pending): admin runs <code>enroll_puf()</code> → mask/ECC helper data written to ESP32 NVS → server verifier registration via <code>register_puf_node.py</code> → per-boot reconstruction and nonce-bound mutual identification confirmed over $N = 30$ real hard-reset trials (30/30 success, Table 19)
6.2.1	Functional test coverage	8 tests: boot (FT-01), WiFi from NVS (FT-02), P2 handshake (FT-03), P3 session (FT-04), P4 chain (FT-05), fuzzy EMS (FT-06), ADC saturation (FT-07), degraded mode (FT-08); 100% pass rate
6.2.2	Tamper verification	NVS parameter tampering confirmed as authentication failure (single targeted negative test; full penetration testing deferred to ITER 2)
7.1.1	Monitoring (Stage 7.1.1)	Per-node real-time: V_{bus} , I_{bus} , P_{bus} , V_{gen} , I_{gen} , P_{gen} , D_{pwm} ; authentication status; session history (evidentiary log, no cryptographic chain under the PUF-based redesign, see Limitations); heap free
7.1.4	Maintenance cycles (historical, prior PSK+NVS/MicroPython scheme)	Three maintenance moments during Stages 5–6 of the earlier instantiation: (i) HKDF byte slice bug in P3 (M2: code revision); (ii) static nonce in session confirmation, MicroPython default RNG insufficient (M1: technology re-evaluation); (iii) NVS hex encoding error in provisioning (M2: code revision). M3 (architectural redesign) not triggered at TRL 3. Retained as a documented finding from the prior instantiation. PUF-based redesign (SMG_NODE_01, Section 6.4): one maintenance moment, an enrollment-state-confusion bug causing an unbounded re-enrollment loop on reconstruction retry, identified and fixed (M1) prior to the $N = 30$ bench trial; SMG_NODE_02 maintenance-cycle evidence remains pending.
7.2.1	Anomaly detection	Per-node trust score (float 0.0–1.0): initialized 0.80 on successful P2 identification; -0.30 on P3 authentication-tag failure; -0.40 on V_{bus} anomaly; UNTRUSTED (< 0.30) ⇒ TCP dropped; recovery requires re-auth
7.2.2	Update elements	OTA update requires admin token + firmware signature (deferred to ITER 2)
7.2.3	Incident response	Auth failures logged; > 3 consecutive failures ⇒ quarantine
7.2.4	Security-aware EMS	Global fuzzy EMS (27 rules) on RPi4 accepts the per-node trust score from 7.2.1 as a third input alongside voltage tracking error and bus current, producing security-aware energy management without a separate intrusion response

- Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., Zhou, Y., 2017. Understanding the Mirai botnet, in: Proceedings of the 26th USENIX Security Symposium, USENIX Association, Vancouver, BC, Canada. pp. 1093–1110.
- Ayala, I., Amor, M., Fuentes, L., Troya, J.M., 2015. A software product line process to develop agents for the IoT. *Sensors* 15, 15640–15660.
- Bakiris, E., Oikonomakis, G., 2025. IoT-driven security solutions, in: Cybersecurity: 9th European Interdisciplinary Cybersecurity Conference, EICC 2025, Rennes, France, June 18–19, 2025, Proceedings, Springer Nature. p. 275.
- Blanco, C., Rosado, D.G., Varela-Vaca, Á.J., Gómez-López, M.T., Fernández-Medina, E., 2023. Onto-CARMEN: Ontology-driven approach for Cyber-Physical System security requirements meta-modelling and reasoning. *Internet of Things* 24, 100989.
- Campos-Sanchez, R., Feregrino-Urbe, C., Hernandez-Martinez, L., Penuelas-Angulo, A., 2026. An identity-centric paradigm for the internet of things: Security, trust, and sovereignty by design. *IoT Submitted for publication*.
- Casadei, R., Tsigkanos, C., Viroli, M., Dustdar, S., 2019. Engineering resilient collaborative edge-enabled IoT, in: 2019 IEEE International Conference on Services Computing (SCC), IEEE. pp. 36–45.
- Cicirelli, F., Fortino, G., Guerrieri, A., Spezzano, G., Vinci, A., 2017. Meta-modeling of smart environments: from design to implementation. *Advanced Engineering Informatics* 33, 274–284.
- Cloud Security Alliance, 2023. IoT Security Controls Framework, Version 3. Technical Report. Cloud Security Alliance. URL: <https://cloudsecurityalliance.org/research/working-groups/internet-of-things>.
- Costa, B., Pires, P.F., Delicato, F.C., 2016. Modeling IoT applications with SysML4IoT, in: 2016 42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE. pp. 157–164.

- European Telecommunications Standards Institute, 2020. ETSI EN 303 645 V2.1.1: Cyber security for consumer internet of things: Baseline requirements. European Standard. URL: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/02.01.01_60/en_303645v020101p.pdf.
- European Union Agency for Cybersecurity, 2017. Baseline Security Recommendations for IoT in the Context of Critical Information Infrastructures. Technical Report. European Union Agency for Cybersecurity (ENISA).
- Fagan, M., Marron, J., Brady, K., Cuthill, B., Megas, K., Herold, R., 2021. IoT Device Cybersecurity Guidance for the Federal Government: Establishing IoT Device Cybersecurity Requirements. NIST Special Publication 800-213. National Institute of Standards and Technology (NIST). Gaithersburg, MD. doi:[10.6028/NIST.SP.800-213](https://doi.org/10.6028/NIST.SP.800-213).
- Fernandez, E.B., Brazhuk, A., 2024. A critical analysis of Zero Trust Architecture (ZTA). *Computer Standards & Interfaces* 89, 103832.
- Fernando, Y., Tseng, M.L., Wahyuni-Td, I.S., de Sousa Jabbour, A.B.L., Chiappetta Jabbour, C.J., Foropon, C., 2023. Cyber supply chain risk management and performance in industry 4.0 era: information system security practices in Malaysia. *Journal of Industrial and Production Engineering* 40, 102–116.
- Fortino, G., Russo, W., Savaglio, C., Shen, W., Zhou, M., 2017. Agent-oriented cooperative smart objects: From IoT system design to implementation. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 48, 1939–1956.
- Fortino, G., Savaglio, C., Spezzano, G., Zhou, M., 2020. Internet of things as system of systems: A review of methodologies, frameworks, platforms, and tools. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 51, 223–236.
- Guerrero-Ulloa, G., Rodriguez-Dominguez, C., Hornos, M.J., 2023. Agile methodologies applied to the development of Internet of Things (IoT)-based systems: A review. *Sensors* 23, 790.
- ISO/IEC, 2019. ISO/IEC 9798-3:2019 information technology — security techniques — entity authentication — part 3: Mechanisms using digital

- signature techniques. International Organization for Standardization. 5th edition.
- Javanmardi, S., Shojafar, M., Mohammadi, R., Alazab, M., Caruso, A.M., 2023. An SDN perspective IoT-fog security: A survey. *Computer Networks* 229, 109732.
- Josyula, S.K., Gupta, D., 2017. A new security methodology for internet of things, in: 2017 International Conference on Computing, Communication and Automation (ICCCA), IEEE. pp. 613–618.
- Licata, J., McWhite, R., Calloway, L., Gilbert, D., Anderson, M., Snyder, J., Miller, J., 2025. Developing Security, Privacy, and Cybersecurity Supply Chain Risk Management Plans for Systems. Technical Report. National Institute of Standards and Technology.
- Makhdoom, I., Abolhasan, M., Lipman, J., Liu, R.P., Ni, W., 2018. Anatomy of threats to the internet of things. *IEEE Communications Surveys & Tutorials* 21, 1636–1675.
- Manate, B., Forti, F., Moore, P., 2014. Applying the Prometheus methodology for an Internet of Things architecture, in: 2014 IEEE/ACM 7th International Conference on Utility and Cloud Computing, IEEE. pp. 435–442.
- Merzouk, S., Cherkaoui, A., Marzak, A., Nawal, S., 2020. IoT methodologies: comparative study. *Procedia Computer Science* 175, 585–590.
- Moreta, N., Aragon, D., Ona, S., Jaramillo, A., Ibarra, J., Jahankhani, H., 2023. Comparison of cybersecurity methodologies for the implementing of a secure IoT architecture, in: *Cybersecurity in the Age of Smart Societies: Proceedings of the 14th International Conference on Global Security, Safety and Sustainability*, London, September 2022, Springer. pp. 9–29.
- Morin, B., Harrand, N., Fleurey, F., 2017. Model-based software engineering to tame the IoT jungle. *IEEE Software* 34, 30–36.
- Muñoz, M., Torres, M., Gil, J.D., Guzmán, J.L., 2025. An Internet of Things platform for heterogeneous data integration: Methodology and application examples. *Journal of Network and Computer Applications* 240, 104197.

- Okika, N., Nwatuze, G.A., Odozor, L., Oni, O., Idoko, I.P., 2025. Addressing IoT-driven cybersecurity risks in critical infrastructure to safeguard public utilities and prevent large-scale service disruptions. *International Journal of Innovative Science and Research Technology* 10.
- Patel, P., Cassou, D., 2015. Enabling high-level application development for the Internet of Things. *Journal of Systems and Software* 103, 62–84.
- Ray, R.K., Chowdhury, F.R., Hasan, M.R., 2024. Blockchain applications in retail cybersecurity: Enhancing supply chain integrity, secure transactions, and data protection. *Journal of Business and Management Studies* 6, 206–214.
- Saqib, M., Moon, A.H., 2023. A systematic security assessment and review of Internet of Things in the context of authentication. *Computers & Security* 125, 103053.
- Sassalou, E.N., Vasileiadis, S., Kazazis, S.A., Protogerou, G., Varvitsiotis, N., Karas, D.S., Giannetsos, T., Tsintzos, S., 2025. A PUF-based root-of-trust for resource-constrained IoT devices, in: 2025 IEEE International Conference on Cyber Security and Resilience (CSR), IEEE. pp. 824–831.
- Savaglio, C., Fortino, G., Gravina, R., Russo, W., 2018. A methodology for integrating internet of things platforms, in: 2018 IEEE International Conference on Cloud Engineering (IC2E), IEEE. pp. 317–322.
- Shahinzadeh, G., Shahinzadeh, H., Tanwar, S., 2024. Security and privacy issues in the Internet of Things: A comprehensive survey of protocols, standards, and the revolutionary role of blockchain, in: 2024 8th International Conference on Smart Cities, Internet of Things and Applications (SCIoT), IEEE. pp. 59–67.
- Shostack, A., 2014. *Threat Modeling: Designing for Security*. Wiley, Indianapolis, IN.
- Sorri, K., Mustafee, N., Seppanen, M., 2022. Revisiting IoT definitions: A framework towards comprehensive use. *Technological Forecasting and Social Change* 179, 121623.
- Spanoudakis, N., Moraitis, P., 2015. Engineering ambient intelligence systems using agent technology. *IEEE Intelligent Systems* 30, 60–67.

- Stanicek, O., 2022. esp32_puflib: An SRAM-based physical unclonable function library for the ESP32 microcontroller family. Software library, ESP-IDF component, https://github.com/staniond/esp32_puflib.
- Turner, S.W., Karakus, M., Guler, E., Uludag, S., 2023. A promising integration of SDN and blockchain for IoT networks: A survey. *IEEE Access* .
- UK Government Security, 2024. Secure by design principles. <https://www.security.gov.uk/guidance/secure-by-design/principles/>. Accessed: 30 April 2024.
- Visintin, A., 2024. Secure IoT and the Identity of Things. Ph.D. thesis. Università degli Studi di Padova.
- Yin, R.K., 2014. Case Study Research: Design and Methods. 5th ed., SAGE Publications, Thousand Oaks, CA.
- Zanasi, C., Russo, S., Colajanni, M., 2024. Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures. *Ad Hoc Networks* , 103414.
- Zhou, I., Makhdoom, I., Shariati, N., Raza, M.A., Keshavarz, R., Lipman, J., Abolhasan, M., Jamalipour, A., 2021. Internet of things 2.0: Concepts, applications, and future directions. *IEEE Access* 9, 70961–71012.